MedInsight: An Explainable Machine Learning System for Stroke Risk Prediction and Personalised Health Screening

A Project Report

In the partial fulfillment of the requirements for the award of the degree of

BCA

at

Ardent Computech Pvt. Ltd

Submitted by

PRIYODARSHINI BANERJEE (GNIT/2024/0013)

GURU NANAK INSTITUTE OF TECHNOLOGY

Certificate from the Mentor

This is to certify that **PRIYODARSHINI BANERJEE** has completed the project **MedInsight: An Explainable Machine Learning System for Stroke Risk Prediction and Personalised Health Screening** under my supervision during the period from **JULY, 2026** to **AUGUST, 2026** which is in partial fulfillment of requirements for the award of the **BCA** and submitted to Department **COMPUTER APPLICATIONS** of **GURU NANAK INSTITUTE OF TECHNOLOGY.**

Signature of the Mentor

Date: 26.08.2026

Acknowledgment

I take this opportunity to express my deep gratitude and sincerest thanks to my project mentor, **SK ANARUL** for giving the most valuable suggestions, helpful guidance, and encouragement in the execution of this project work.

I would like to give a special mention to my colleagues. Last but not least I am grateful to all the faculty members of the **Ardent Computech Pvt. Ltd.** for their support.

ABSTRACT

Background:

Stroke is a major health condition that can lead to serious long-term disability and mortality, making early identification of potential risk an important aspect of preventive healthcare. Traditional risk assessment depends on clinical evaluation and multiple health indicators, which can make large-scale screening time consuming. Machine Learning provides an approach for analysing patient-related demographic, health, and lifestyle factors to identify patterns associated with stroke occurrence. MedInsight is developed as a Machine Learning-based screening and decision-support system that estimates an individual's stroke risk and provides interpretable insights into the factors influencing the prediction.

Problem Statement and Objective:

The identification of individuals who may have an increased likelihood of stroke requires consideration of multiple interacting risk factors such as age, hypertension, heart disease, glucose level, BMI and lifestyle characteristics. A major challenge in developing a reliable prediction system is the significant imbalance between stroke and non-stroke cases in the dataset. This project aims to develop a Machine Learning-based stroke risk prediction system that can handle class imbalance while prioritising the identification of potential stroke cases. The objective is to build an accessible and interpretable screening application that not only provides a risk prediction but also explains the major factors contributing to the model's decision.

Dataset:

The project uses a stroke prediction dataset containing **5110 patient records** with demographic, health and lifestyle-related attributes. The dataset includes features such as age, gender, marital status, work type, residence type, smoking status, hypertension, heart disease, average glucose level and BMI, with **stroke** as the target variable. The dataset contains substantially more non-stroke cases than stroke cases, creating a class imbalance problem that was considered during model development and evaluation.

Methodology:

The MedInsight system was developed using **Python, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, SHAP and Streamlit**. The dataset was first explored and visualised to understand the distribution of stroke cases and relationships among numerical features. Data preprocessing included handling categorical variables through encoding, removal of the patient identifier, separation of input features and target variable and feature scaling where required. The dataset was divided into training and testing sets using an **80:20 ratio**. Multiple Machine Learning models including **Logistic Regression, K-Nearest Neighbours (KNN), Decision Tree, Random Forest, SMOTE-based Logistic Regression and Balanced Logistic Regression**, were trained and evaluated using accuracy, precision, recall and F1-score. Considering the importance of identifying potential stroke cases, **recall** was given particular importance during model selection. A **Tuned Balanced Logistic Regression** model was selected as the final deployment model. To improve interpretability **SHAP (SHapley Additive exPlanations)** was incorporated to explain both overall feature importance and individual predictions. The final model was integrated into a Streamlit-based application that provides patient screening, risk assessment, model analytics and personalised health guidance.

Evaluation and Results:

The performance of the developed Machine Learning models was evaluated using Accuracy, Precision, Recall and F1-score on the testing dataset. Logistic Regression, K-Nearest Neighbours (KNN), Decision Tree, Random Forest, SMOTE-based Logistic Regression and Balanced Logistic Regression were trained and compared. Since stroke cases are substantially fewer than non-stroke cases, recall was given particular importance during model evaluation. The Balanced Logistic Regression model achieved a recall of **80%**, indicating its stronger ability to identify patients belonging to the stroke class compared with the other evaluated models. The final tuned Balanced Logistic Regression model was selected for deployment because the project prioritises reducing missed potential stroke cases. The model achieved an **ROC-AUC score of 0.828**, demonstrating useful discriminatory capability between the two outcome classes.

Prediction System:

The trained Machine Learning model was integrated into a **Streamlit-based web application** named **MedInsight** to provide an interactive stroke-risk screening system. Users can enter relevant demographic, health and lifestyle information after which the system processes the inputs and generates an estimated stroke-risk prediction. The application presents the prediction together with personalised health guidance based on relevant risk factors. A dedicated **Model Analytics** section provides model-performance information, including model comparison, recall comparison, confusion-matrix analysis, correlation analysis and explainability through SHAP. The system also supports the generation of a personalised screening report. MedInsight is designed as a screening and decision-support tool and does not replace professional medical diagnosis or treatment.

Keywords:

Stroke Risk Prediction, Machine Learning, Explainable AI, Logistic Regression, Balanced Logistic Regression, SHAP, Class Imbalance, Streamlit, Healthcare Analytics, Risk Screening, Personalised Health Guidance, Decision Support.

TABLE OF CONTENTS

1. **Introduction 8**
    - 1.1 Background of Stroke Risk Prediction **.....8**
    - 1.2 Importance and Relevance**..... 9**
    - 1.3 Problem Statement **.....10**
    - 1.4 Objectives of the Project **.....10**
    - 1.5 Scope of the Project**..... 10-11**
    - 1.6 Significance of the Project **.....11**
2. **Literature Review…… 12-16**
3. **System Analysis…….17-20**
4. **System Design……21-25**
5. **Methodology…….. 26-37**
6. **Algorithms and Techniques……..38-51**
7. **Implementation……..52-74**
8. **Dataset and Data Visualization……75-83**
9. **Result and Discussion…….... 84-92**
10. **Advantages and Limitations…….... 93-94**
11. **Conclusion…….. 95-97**
12. **Future Scope ……..98-99**
13. **References……...100**

CHAPTER 1 – INTRODUCTION

1.1 Background of Stroke Risk Prediction

Stroke is a serious health condition that occurs when the blood supply to a part of the brain is interrupted or reduced, potentially resulting in long-term disability or other severe health consequences. The risk of stroke is influenced by multiple demographic, health and lifestyle-related factors, including age, hypertension, heart disease, average glucose level, body mass index (BMI) and smoking status. Since these factors may interact in complex ways, identifying individuals who may have an increased likelihood of stroke can be challenging through simple assessment of individual factors alone.

The availability of healthcare datasets has created opportunities to apply Machine Learning techniques for analysing patterns within patient information. Machine Learning classification models can learn relationships between patient characteristics and previously observed stroke outcomes and can subsequently be used to estimate the likelihood of stroke for new cases. Such systems can support preliminary screening by providing a consistent, data-driven assessment of risk factors.

However stroke prediction presents an important challenge because stroke datasets commonly contain considerably fewer stroke cases than non-stroke cases. This class imbalance can cause a model to favour the majority class and potentially miss individuals belonging to the stroke class. Therefore model evaluation should consider metrics such as recall, precision and F1-score rather than relying only on accuracy.

To address these considerations MedInsight was developed as an explainable Machine Learning based stroke risk screening and decision support system. The system evaluates relevant patient information, applies a trained predictive model and provides an estimated stroke-risk outcome. It further incorporates SHAP-based explainability to provide insight into the factors influencing individual predictions. Through its Streamlit-based interface, MedInsight combines prediction, model analytics and personalised health guidance within a single screening-oriented application.

Importance and Relevance

- **Early stroke risk identification** is important because recognising individuals with elevated risk factors can support timely preventive attention and further clinical assessment.
- **Machine Learning-based prediction** can assist in analysing multiple demographic, health and lifestyle factors simultaneously and identifying patterns associated with stroke occurrence.
- **Class-imbalance handling** is particularly relevant in stroke prediction because stroke cases are substantially fewer than non-stroke cases, making it important to evaluate the model using metrics beyond accuracy.
- **Explainable Artificial Intelligence (XAI)** improves the usefulness of prediction systems by helping users understand which patient characteristics contribute to the model's prediction.
- **Integrated healthcare screening systems** can combine prediction, model analytics and personalised guidance in a single application, making the technology more accessible for preliminary risk assessment.
- **Development of MedInsight** demonstrates a complete Machine Learning workflow involving data preprocessing, exploratory data analysis, model training, evaluation, explainability using SHAP and deployment through a Streamlit-based application.

1.2 Problem Statement

Stroke prediction using patient health and lifestyle information presents several challenges. The dataset contains a large imbalance between non-stroke and stroke cases, which can cause conventional classification models to favour the majority class. As a result a model may achieve high overall accuracy while still failing to identify a significant proportion of actual stroke cases.

Another challenge is the interpretability of Machine Learning predictions. For a healthcare-oriented screening system simply producing a predicted outcome is not sufficient ; users should also be able to understand the important factors influencing the prediction.

Therefore the problem addressed by this project is to develop a Machine Learning-based stroke risk screening system that can handle class imbalance, prioritise the identification of

potential stroke cases, provide reliable model evaluation and offer interpretable insights into individual predictions.

1.3 Objectives of the Project

The main objectives of the MedInsight project are:

1. To develop a **Machine Learning-based stroke risk prediction system** using relevant patient demographic, health and lifestyle features.
2. To perform **data preprocessing and exploratory data analysis** to identify patterns, distributions and relationships within the stroke dataset.
3. To address the **class imbalance** present in the dataset and evaluate suitable classification models.
4. To compare different Machine Learning models using **Accuracy, Precision, Recall and F1-score** with particular importance given to recall for identifying potential stroke cases.
5. To select and deploy a **Tuned Balanced Logistic Regression** model as the final prediction model.
6. To incorporate **SHAP-based Explainable AI** to identify and communicate the factors influencing model predictions.
7. To develop an interactive **Streamlit-based screening application** that provides risk prediction, model analytics and personalised health guidance.
8. To provide a **screening and decision-support tool** that can assist users in understanding potential stroke risk while not replacing professional medical diagnosis.

1.4 Scope of the Project

The scope of this project is to develop a Machine Learning-based stroke risk screening and decision support system using demographic, health and lifestyle related patient information. The project includes data preprocessing, exploratory data analysis, handling of class imbalance, training and comparison of multiple classification models, model evaluation and selection of a suitable final model. It also includes the use of SHAP for model explainability and the integration of the trained

model into a Streamlit-based web application providing patient screening, model analytics and personalised health guidance.

The project is intended for **preliminary risk screening and educational decision support** and does not provide medical diagnosis or treatment recommendations. It does not include clinical validation, integration with hospital systems, real-time patient monitoring or replacement of professional medical judgement. These areas may be considered as potential extensions of the system in future work.

1.5 Significance of the Project

A Machine Learning-based stroke risk screening system can be significant to different stakeholders:

- **Individuals/Patients:** The system can provide an early indication of potential stroke risk and encourage individuals to seek appropriate professional medical assessment when necessary.
- **Healthcare Professionals:** MedInsight can serve as a data-driven decision-support aid by presenting predicted risk along with the factors influencing the prediction.
- **Researchers and Students:** The project demonstrates the application of Machine Learning, class-imbalance handling, model evaluation and Explainable AI techniques to healthcare-related data.
- **Healthcare Analytics:** The system demonstrates how predictive analytics can be integrated with an interactive application to support preliminary health risk screening and interpretation.
- **Machine Learning Development:** The project provides a practical implementation of the complete workflow from data preprocessing and exploratory analysis to model comparison, explainability, and Streamlit deployment.

It is important to note that MedInsight is intended for preliminary risk screening and educational decision support and should not be considered a substitute for professional medical diagnosis or treatment.

CHAPTER 2 - LITERATURE REVIEW

2.1 Machine Learning in Healthcare

Machine Learning has become an important component of healthcare analytics because of its ability to analyse patient-related data and identify patterns associated with different health outcomes. It is applied in areas such as disease-risk prediction, patient classification, medical data analysis and decision support.

Healthcare datasets often contain demographic, clinical and lifestyle related information. Machine Learning classification models can analyse these factors together and learn patterns from previously observed outcomes to generate predictions for new cases. However healthcare data may present challenges such as missing values, categorical variables, class imbalance and the need for interpretable predictions.

In the context of stroke risk prediction Machine Learning can therefore support preliminary screening by analysing multiple risk factors simultaneously. MedInsight applies this approach through data preprocessing, exploratory analysis, classification-model comparison, class imbalance handling, recall-focused evaluation and SHAP-based explainability, integrated within an interactive screening application.

2.2 Machine Learning for Stroke Risk Prediction

Stroke risk prediction is an important application of Machine Learning because stroke outcomes can be associated with a combination of demographic, medical, and lifestyle-related factors. Variables such as age, hypertension, heart disease, glucose level, BMI, and smoking behaviour can provide useful information for analysing patterns associated with stroke occurrence.

Machine Learning classification methods allow multiple patient attributes to be considered together rather than evaluating individual risk factors independently. Different classification approaches can therefore be used to estimate the likelihood of belonging to a stroke or non-stroke outcome class based on patterns learned from existing patient data.

An important challenge in stroke prediction is the **unequal distribution of stroke and non-stroke observations**. This can influence model learning and may make overall accuracy

insufficient as the only measure of performance. Therefore, evaluation measures such as **precision, recall, F1-score and ROC-AUC** are useful when assessing stroke-prediction models particularly when correctly identifying positive stroke cases is important.

2.3 Classification Algorithms Used in the Project

To develop and evaluate the stroke-risk prediction system multiple classification algorithms were considered to compare their predictive performance. The algorithms used in the project include:

- **Logistic Regression:** A statistical classification technique that estimates the probability of an observation belonging to a particular class. It provides a relatively interpretable baseline for binary classification.
- **K-Nearest Neighbours (KNN):** A distance based algorithm that classifies a new observation according to the classes of its nearest neighbouring observations.
- **Decision Tree:** A tree based classification method that makes predictions through a sequence of feature based decisions.
- **Random Forest:** An ensemble learning method that combines multiple decision trees to produce a more robust classification result.
- **SMOTE-based Logistic Regression:** Logistic Regression combined with **Synthetic Minority Over-sampling Technique (SMOTE)** to address the imbalance between the stroke and non-stroke classes during model training.
- **Balanced Logistic Regression:** A Logistic Regression approach using class balancing so that the minority stroke class receives greater consideration during model learning.

The performance of these approaches was compared using **Accuracy, Precision, Recall and F1-score**. Since identifying potential stroke cases is particularly important in this project, **Recall** was given additional importance during model selection. The tuned version of Balanced Logistic Regression was ultimately selected as the final deployment model.

2.4 Role of Patient Risk Factors in Stroke Prediction

The major patient-related factors considered in stroke-risk prediction include:

- **Age:** Increasing age can be an important factor associated with stroke occurrence and is one of the key demographic variables considered by the prediction model.
- **Hypertension:** The presence of hypertension represents an important health related risk factor and can contribute to the model's assessment of stroke risk.
- **Heart Disease:** Existing heart disease is another relevant medical characteristic that may be associated with increased stroke risk.
- **Average Glucose Level:** Blood glucose level provides physiological information that can contribute to identifying patterns associated with stroke outcomes.
- **BMI:** Body Mass Index provides information related to an individual's weight status and is considered among the health related features in the dataset.
- **Smoking Status:** Smoking behaviour represents a lifestyle related factor that may contribute to stroke-risk patterns.
- **Other Demographic and Lifestyle Features:** Variables such as gender, marital status, work type and residence type provide additional information that can be considered collectively by the Machine Learning model.
- **Feature Contribution and Explainability:** The relative contribution of different features can vary across predictions. **SHAP-based analysis** is therefore used in MedInsight to provide insight into how individual features influence model predictions.

2.5 Comparative Analysis of Existing Approaches

Aspect	Traditional Approach	Machine Learning Approach
Basis of Assessment	Clinical examination, medical history, and individual risk factor assessment	Multiple patient attributes analyzed using Machine Learning models
Risk Analysis	Individual risk factors may be assessed separately	Multiple demographic, medical and lifestyle factors can be analyzed together
Prediction	Depends mainly on professional assessment	Automated prediction based on trained classification models
Class Imbalance	Not generally considered as a modelling issue	Can be addressed using techniques such as SMOTE and class balancing
Performance Evaluation	Primarily based on clinical assessment and established measures	Accuracy, Precision, Recall, F1-score and ROC-AUC can be used
Interpretability	Based on clinical reasoning and medical expertise	Can be supported using Explainable AI techniques such as SHAP

Machine Learning therefore provides a data-driven approach for analysing multiple stroke-related factors and generating consistent predictions, while explainability techniques can improve understanding of model decisions.

2.6 Research Gap

Based on the analysis of existing approaches and the dataset used in this project, several research gaps and challenges remain relevant to Machine Learning-based stroke-risk prediction:

- **Dataset Limitation:** The project relies on a single dataset, which may limit the generalizability of the developed model to different patient populations and healthcare settings.
- **Limited Clinical Features:** The dataset contains selected demographic, medical and lifestyle-related attributes but does not include other potentially relevant clinical information such as detailed medical history, laboratory investigations, imaging data or genetic information.
- **Model Generalization:** The model is evaluated using the available training and testing data. Further validation using independent datasets and broader populations would be required to establish stronger generalization.
- **Clinical Validation:** MedInsight is designed as a screening and decision support system and has not undergone clinical validation with real world healthcare professionals or clinical environments.
- **Need for Larger and More Diverse Datasets:** Larger datasets containing patients from different age groups, geographic regions and healthcare settings could improve the robustness and reliability of stroke-risk prediction models.
- **External Validation:** Future research can evaluate the selected model on external datasets to determine whether its performance remains consistent beyond the dataset used for development.

  This project does not claim to solve all of these research challenges. Instead it demonstrates an integrated Machine Learning-based stroke-risk screening approach that incorporates class imbalance handling, model evaluation and SHAP-based explainability while highlighting limitations and possible directions for future research.

CHAPTER 3 – SYSTEM ANALYSIS

3.1 Existing System

The existing approach to stroke-risk assessment primarily relies on clinical examination, medical history and professional evaluation of individual risk factors. Healthcare professionals may consider factors such as age, hypertension, heart disease, glucose level, BMI, smoking status and other patient characteristics while assessing an individual's potential risk of stroke.

Traditional assessment can provide valuable clinical insight; however it may depend on the availability of healthcare professionals and the time required to evaluate multiple patient factors. The assessment is also primarily based on clinical judgement and established medical knowledge rather than an automated analysis of patterns across large patient datasets.

With the increasing availability of healthcare data Machine Learning-based approaches provide an alternative method for analysing multiple patient attributes simultaneously. Such approaches can support preliminary stroke-risk screening by learning patterns from previously observed patient data.

3.2 Problems with Existing Approaches

- Requires professional evaluation and can be time consuming.
- Analysing several patient factors together can be complex.
- Fewer stroke cases may cause models to favour the majority class.
- Traditional approaches do not provide automated preliminary screening.
- Predictions may not clearly explain the factors influencing the outcome.
- Prediction, analytics, explainability and guidance may not be available in one system.

3.3 Proposed Machine Learning System

The proposed MedInsight system uses Machine Learning techniques to predict stroke risk from demographic, medical and lifestyle-related patient information. The dataset is

preprocessed and prepared for model development, including appropriate handling of categorical variables, feature scaling and class imbalance. Multiple classification models are trained and evaluated using **Accuracy, Precision, Recall and F1-score**. Considering the importance of identifying potential stroke cases, recall is given particular importance during model comparison and the **Tuned Balanced Logistic Regression** model is selected as the final model.

The selected model is integrated into a **Streamlit-based web application** allowing users to enter patient information and receive an estimated stroke risk outcome. The application also provides model analytics, SHAP-based explanations and personalised health guidance, making MedInsight an interactive screening and decision support system.

3.4 Advantages of Proposed Approach

- **Automated Screening:** Provides an automated preliminary assessment of stroke risk.
- **Multiple Factor Analysis:** Considers demographic, medical and lifestyle features together.
- **Class-Imbalance Handling:** Uses balancing techniques to give greater consideration to the minority stroke class.
- **Recall Focused Evaluation:** Gives particular importance to recall when comparing model performance.
- **Explainable Predictions:** Uses **SHAP** to show the factors influencing model predictions.
- **Interactive Application:** Provides patient screening and model analytics through a **Streamlit** interface.
- **Personalised Guidance:** Generates guidance based on the patient's entered information and identified risk factors.
- **Integrated System:** Combines prediction, analytics, explainability and screening support in a single application.

3.5 Technical Requirements

- The stroke dataset is loaded using **Pandas** and preprocessed by separating the input features and target variable.

- The dataset is divided into **training and testing sets** and **StandardScaler** is applied to the numerical features where required.
- Multiple classification models including **Logistic Regression, KNN, Decision Tree, Random Forest, SMOTE-based Logistic Regression** and **Balanced Logistic Regression**, are trained and evaluated using appropriate performance metrics.
- The trained **Tuned Balanced Logistic Regression** model is integrated into a **Streamlit** web application that accepts patient information and provides the predicted stroke risk outcome.
- The application also incorporates **model analytics, SHAP-based explainability** and **personalised health guidance** to support the screening process.

3.6 Functional Requirements

- The system shall accept relevant **patient demographic, medical and lifestyle information** as input.
- The system shall preprocess the user-provided input in the required format before prediction.
- The system shall apply the trained **Tuned Balanced Logistic Regression** model to the provided patient information.
- The system shall generate and display the predicted **stroke risk outcome**.
- The system shall provide **model analytics**, including comparison of model performance using relevant evaluation metrics.
- The system shall provide a **confusion matrix** for the selected final model.
- The system shall provide **correlation analysis** of relevant numerical features.
- The system shall provide **SHAP-based explanations** to show the factors influencing individual predictions.
- The system shall generate **personalised screening guidance** based on the user's entered information and prediction.
- The system shall provide an interactive and user-friendly interface through **Streamlit.**

3.7 Non-Functional Requirements

- **Performance:** The system shall generate the stroke risk prediction promptly after receiving the user's input.
- **Usability:** The system shall provide a simple, interactive and user-friendly interface through Streamlit.
- **Reliability:** The system shall provide consistent prediction results for the same input data.
- **Interpretability:** The system shall provide understandable information about the factors influencing predictions through SHAP-based explanations.
- **Maintainability:** The system shall be structured so that the Machine Learning model and application components can be updated or improved when required.

3.8 Limitations of the Existing Implementation

- **Dataset Limitation:** The system is developed using a single stroke dataset which may limit generalisation to other populations.
- **Feature Limitation:** The prediction is based on the features available in the dataset and does not include additional clinical information such as medical imaging or detailed laboratory investigations.
- **Clinical Validation:** The system has not been clinically validated and is intended for screening and decision-support purposes only.
- **External Validation:** The current implementation has not been evaluated on independent external datasets.
- **Real-Time Monitoring:** The application does not provide continuous or real time patient health monitoring.
- **Deployment Limitation:** The current implementation is designed as a Streamlit based application and is not integrated with hospital or clinical information systems.
- **Scope for Improvement:** Model performance and generalisation may be further improved using larger and more diverse datasets, additional validation and further model development.

CHAPTER 4 – SYSTEM DESIGN

4.1 Overall System Architecture

The system architecture presents the complete workflow of the MedInsight Stroke-Risk Screening System from dataset preprocessing and model development through the Streamlit application, prediction engine, explainability module and personalised screening guidance.

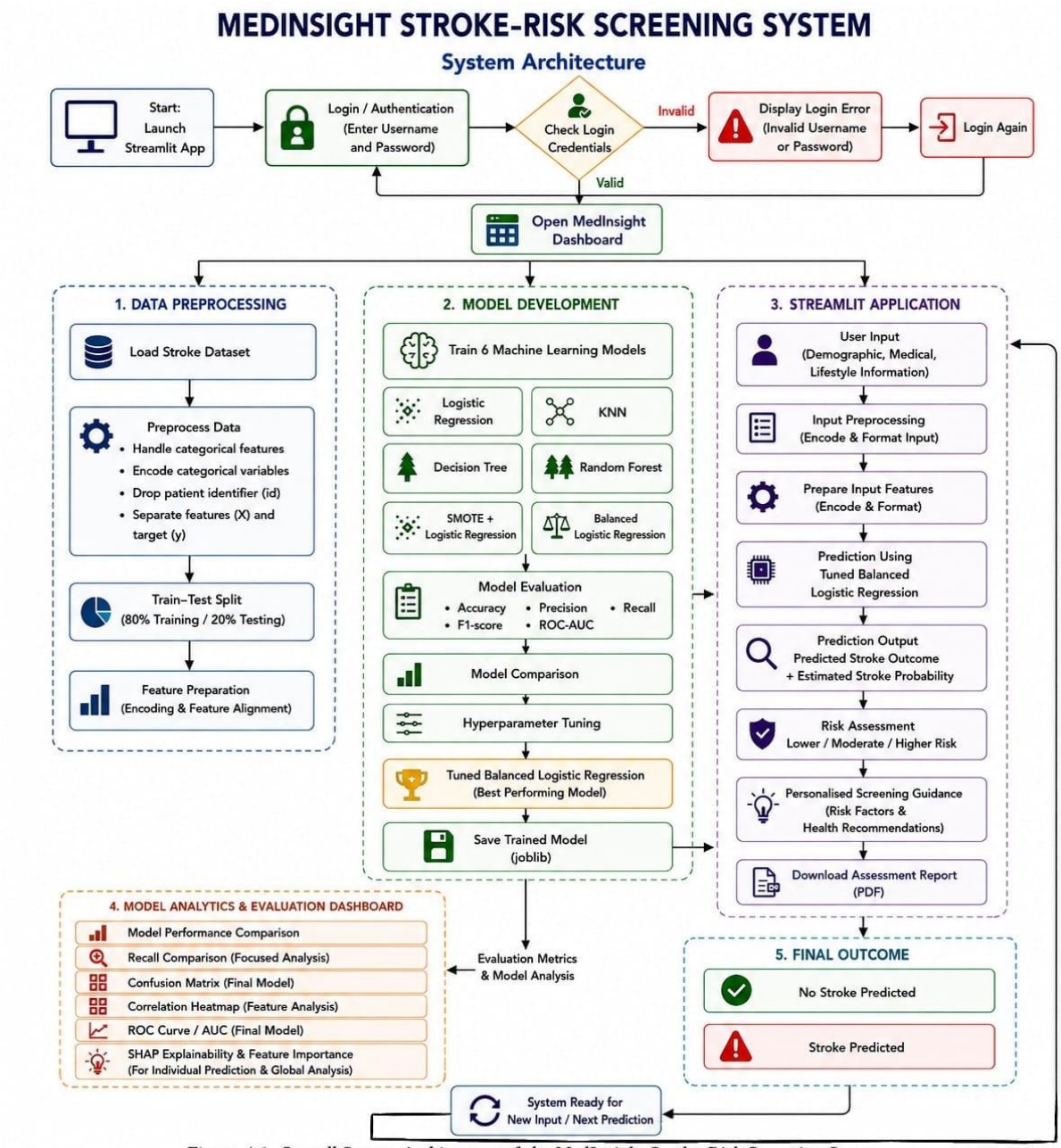


Figure 4.1: Overall System Architecture of the MedInsight Stroke-Risk Screening System

4.2 Data Flow Diagram

The Data Flow Diagram illustrates the movement of data through the MedInsight Stroke Risk Screening System. It shows how patient information is provided as input, processed by the system, analyzed using the trained Machine Learning model and transformed into a predicted stroke outcome. The diagram also represents the flow of data involved in model analytics, explainability and personalised screening guidance.

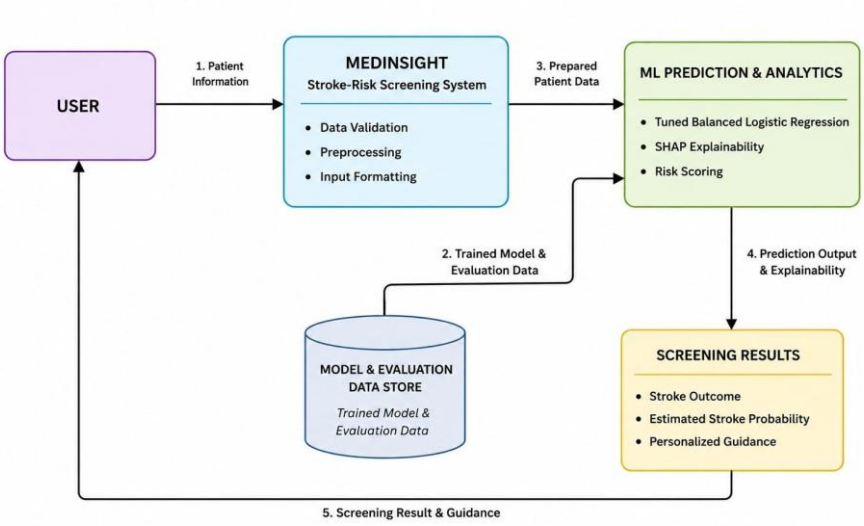


Figure 4.2: Data Flow Diagram of Stroke - Risk Screening System

4.3 System Flowchart

Figure 4.3 illustrates the step by step workflow of the MedInsight Stroke-Risk Screening System from data preprocessing and model development to prediction and final screening results.

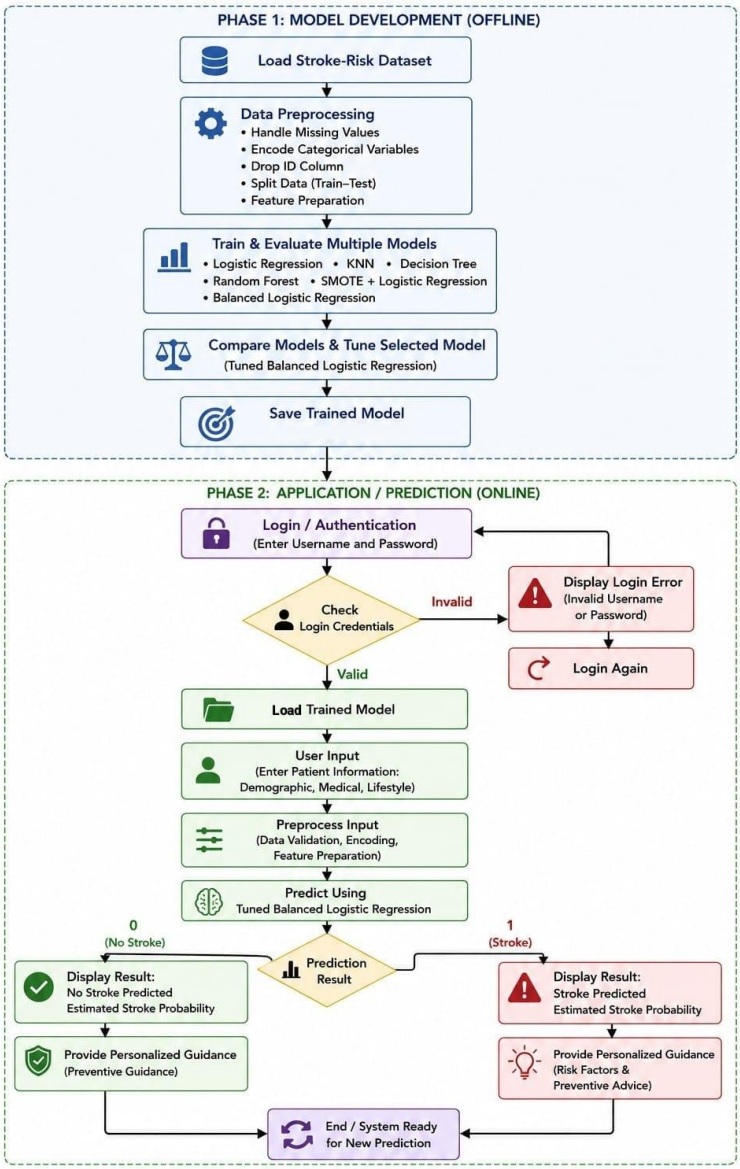


Figure 4.3: System Flowchart of MedInsight Stroke-Risk Screening System

4.4 Module Design

The MedInsight Stroke-Risk Screening System can be conceptually divided into the following functional modules:

- **Login and Authentication Module:** Authenticates users through username and password before providing access to the MedInsight dashboard.
- **Dataset Loading Module:** Loads the stroke-risk dataset for model development and analysis.
- **Data Preprocessing Module:** Handles missing values, encodes categorical variables, removes the patient ID column and prepares the feature set.
- **Data Splitting and Feature Preparation Module:** Splits the dataset into training and testing sets and prepares the features for model training and prediction.

- **Machine Learning Model Development Module:** Trains multiple classification models, including Logistic Regression, KNN, Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression.
- **Model Evaluation and Comparison Module:** Evaluates and compares the trained models using accuracy, precision, recall, F1-score and ROC-AUC.
- **Hyperparameter Tuning and Model Selection Module:** Performs hyperparameter tuning and selects the Tuned Balanced Logistic Regression as the best performing model.
- **Model Saving Module:** Saves the selected trained model as a Joblib file for use during prediction.
- **Prediction Module:** Accepts patient information, preprocesses the input and generates the predicted stroke outcome along with the estimated stroke probability.
- **Risk Assessment Module**: Interprets the predicted probability to provide an appropriate stroke-risk assessment.
- **Explainability and Analytics Module:** Provides model performance analysis, recall comparison, confusion matrix, ROC-AUC analysis, correlation analysis and SHAP-based feature importance. The architecture specifically represents these analytics components.
- **Personalised Guidance Module:** Provides screening guidance based on the predicted outcome and relevant patient risk factors.
- **Assessment Report Module:** Generates a downloadable PDF assessment report containing the screening information and results.
- **Streamlit User Interface Module:** Provides the interactive interface for user authentication, patient information entry, prediction results, risk assessment, guidance and report generation.

4.5 Input and Output Design

Input:

The MedInsight Stroke-Risk Screening System accepts patient information through the Streamlit user interface. The input consists of relevant **demographic, medical and lifestyle attributes** required by the trained machine learning model. The entered data is validated,

encoded, formatted and prepared consistently with the preprocessing applied to the training data before being passed to the selected prediction model.

Output:

The system generates a **stroke-risk prediction** using the **Tuned Balanced Logistic Regression** model. The output indicates whether **stroke is predicted or no stroke is predicted** and displays the **estimated stroke probability**. The system also provides **personalised screening guidance based on the predicted outcome and relevant risk factors** through the Streamlit interface.

Design Element	Description
Primary Input	Patient demographic, medical and lifestyle information entered through the Streamlit interface.
Preprocessed Input	Validated, encoded, formatted and scaled patient features prepared for model prediction.
Excluded Column	Patient ID is excluded because it is an identifier and does not contribute to prediction.
Target Variable	Stroke outcome: **0 = No Stroke, 1 = Stroke**.
Prediction Model	Tuned Balanced Logistic Regression selected as the best performing model.
Primary Output	Predicted stroke outcome: **No Stroke / Stroke**.
Probability Output	Estimated stroke probability displayed as a percentage.
Guidance Output	Personalised screening guidance based on the predicted outcome and relevant risk factors.
Interface	Streamlit based interactive interface for input and result presentation.

CHAPTER 5 – METHODOLOGY

This chapter explains, step by step, the complete Machine Learning pipeline implemented for the **MedInsight Stroke-Risk Screening System**, using the actual preprocessing, model development, evaluation and prediction procedures followed in the project. The methodology covers dataset preparation, feature processing, model training and comparison, model selection and tuning, followed by integration of the trained model with the Streamlit-based screening application.

Step 1 – Problem Definition

Stroke is a serious medical condition associated with various demographic, medical and lifestyle-related factors. The project aims to develop a machine learning-based screening system that analyzes relevant patient information to predict whether a stroke outcome is likely. The system is designed to provide a predicted stroke outcome and estimated stroke probability, along with personalised screening guidance to support preventive assessment. The stroke prediction task is formulated as a binary classification problem, where 0 represents No Stroke and 1 represents Stroke. Multiple machine learning classification models are trained and evaluated and the selected model is further tuned for use in the final prediction system.

Step 2 – Dataset Loading

The stroke-risk dataset is loaded from the Stroke_prediction.csv file using the Pandas library. The dataset contains patient demographic, medical and lifestyle-related attributes along with the stroke variable, which is used as the target variable for prediction. The loaded dataset is then prepared for subsequent inspection, preprocessing, model training and evaluation.
The dataset is loaded using the following statement:

```
df = pd.read_csv('MedInsight/Stroke_prediction.csv')
```

The loaded DataFrame is then used throughout the machine learning workflow for further processing and model development.

Step 3 – Dataset Inspection

Before applying preprocessing techniques, the code inspects the loaded stroke-risk dataset using several Pandas functions:

- The first few records of the dataset are displayed using df.head() to obtain an initial understanding of the available patient attributes and their values.
- The dimensions of the dataset are examined using df.shape, which provides the number of records and attributes present in the dataset.
- The structure of the dataset is examined using df.info(), which provides the column names, data types and non-null information for each attribute.
- The inspection of the dataset helps identify the different demographic, medical and lifestyle-related attributes and provides the information required for the subsequent preprocessing and model development stages.
- The dataset is inspected for missing values using df.isnull().sum(). This helps identify columns containing missing observations before applying the missing-value handling procedure.
- Descriptive statistical information for the **BMI** attribute is examined using df['bmi'].describe() to understand its count, mean, standard deviation, minimum, maximum and other statistical characteristics.
- The distribution of the target variable is examined using df['stroke'].value_counts() to determine the number of records belonging to the **No Stroke (0)** and **Stroke (1)** classes. This inspection also helps identify the class imbalance present in the dataset.
- The unique values of the categorical attributes, including **gender, ever_married, work_type, Residence_type and smoking_status**, are inspected before categorical encoding is applied.

Code used for dataset inspection

```
print(df.head())
print(df.shape)
print(df.info())
```

```
print(df.isnull().sum())
print(df['bmi'].describe())

print(df.isnull().sum())
print(df['stroke'].value_counts())
```

```
print(df['gender'].unique())
print(df['ever_married'].unique())
print(df['work_type'].unique())
print(df['Residence_type'].unique())
print(df['smoking_status'].unique())
```

Step 4 – Missing Value Handling

The dataset is inspected for missing values before model development. The inspection shows that the **bmi** attribute contains missing observations. Instead of removing the affected records the missing BMI values are replaced using the **median BMI value** of the available records. Median imputation is used to retain the available patient records while providing a representative value for the missing observations.

The following statement is used for missing-value handling:

```
df['bmi'] = df['bmi'].fillna(df['bmi'].median())
```

After imputation, the dataset is checked again using:

```
print(df.isnull().sum())
```

This confirms the missing-value status after the preprocessing operation. The resulting dataset is then used for the subsequent categorical encoding and feature preparation stages.

Stage	Number of Records / Missing Values
Total records before imputation	5110
Missing BMI values before imputation	201
Median BMI used for imputation	28.1
Missing BMI values after imputation	0

A total of **201 missing BMI values** were identified in the dataset. These missing values were replaced using the **median BMI value of 28.1**. After median imputation the BMI column contained **0 missing values** and no records were removed from the dataset.

Step 5 – Categorical Data Encoding

Machine learning algorithms require numerical input so the categorical attributes in the stroke-risk dataset are converted into numerical representations before model training. In this project different encoding techniques are applied according to the nature of each categorical attribute.
The gender attribute is encoded using Label Encoding, while ever_married and Residence_type are converted using predefined binary mappings. The multicategory attributes work_type and smoking_status are transformed using OneHot Encoding. This transformation enables the categorical information to be processed by the machine learning models.

```
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])
print(df['gender'].unique())

df['ever_married'] = df['ever_married'].map({'No': 0 , 'Yes': 1})
print(df['ever_married'].unique())
df['Residence_type']= df['Residence_type'].map({'Rural':0, 'Urban': 1})
print(df['Residence_type'].unique())

#one hot
df = pd.get_dummies(df, columns = ['work_type'] , dtype = int)
print(df.head())
df = pd.get_dummies(df , columns = ['smoking_status'] , dtype = int)
```

Categorical Attribute	Encoding Method	Encoded Representation
gender	Label Encoding	Female → 0, Male → 1, Other → 2
ever_married	Binary Mapping	No → 0, Yes → 1
Residence_type	Binary Mapping	Rural → 0, Urban → 1
work_type	One-Hot Encoding	5 binary feature columns
smoking_status	One-Hot Encoding	4 binary feature columns

The categorical attributes are converted into numerical representations using the respective encoding techniques. Label encoding is applied to gender while binary mappings are used for ever_married and Residence_type. The multi-category attributes work_type and smoking_status are transformed using one-hot encoding, resulting in separate binary feature columns. After encoding the dataset increases from **12 original columns to 19 columns**. The encoded dataset is then used for the subsequent feature preparation and model development stages.

Step 6 - Patient Identifier Removal

After categorical encoding the patient identifier column id is removed because it is used only to identify individual patients and does not provide meaningful predictive information for stroke-risk classification.

```
df = df.drop('id' , axis = 1)
```

The dataset changes from **(5110, 18)** to **(5110, 17)** after removing the identifier column.

Step 7 - Feature and Target Separation

Feature and Target Separation is an important step in the machine learning workflow because it defines the input data used by the models and the output that the models are required to predict. In this project, the dataset is divided into a **Feature Matrix (X)** and a **Target Vector (y)**.
The feature matrix X contains the **17** predictive features remaining after the patient identifier column is removed. During this process the id column was removed as it is a unique identifier with no predictive power. The target vector y is assigned to the stroke column, where **0** represents No Stroke and **1** represents Stroke.

The separation is performed using the following statements:

```
X = df.drop('stroke' , axis=1)
y = df['stroke']
```

Component	Shape
Feature Matrix (X)	(5110, 17)
Target Vector (y)	(5110,)

Step 8 - Train-Test Split

The **Train-Test Split** divides the prepared dataset into two independent subsets for model development and evaluation. The training set is used to train the machine learning models, while the testing set is kept separate and used to evaluate their performance on unseen data.

In this project, the dataset is divided using an **80:20 ratio** with 80% of the records used for training and 20% used for testing.

The dataset is split using the **train_test_split**() function:

```
#train - test - split

X_train , X_test , y_train , y_test = train_test_split(X,y, test_size = 0.2 , random_state = 42, stratify = y)
```

The test_size = 0.2 parameter reserves 20% of the data for testing, while random_state = 42 ensures reproducibility. The stratify=y parameter maintains the proportion of the two stroke classes (0 = No Stroke and 1 = Stroke) in both the training and testing subsets.

Subset	Number of Records	Percentage
Training set (X_train, y_train)	4088	80%
Testing set (X_test, y_test)	1022	20%
Total	5110	100%

Step 9 – Feature Scaling

Feature scaling is performed to bring the numerical input features to a comparable scale before training the machine learning models. In this project StandardScaler is used to standardize the feature values based on the training data.

The scaling is performed using the following statements:

```
#scaling

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

The fit_transform() operation is applied to the training data to calculate the scaling parameters and transform the training features. The same parameters are then applied to the testing data using transform(). This ensures that information from the testing set is not used during the fitting process.
The scaled training and testing feature sets are subsequently used for machine learning model training and evaluation.

Step 10 - Machine Learning Model Training

After feature scaling, multiple machine learning classification models are trained using the prepared training data. The models are selected to provide a comparison of different classification approaches for stroke-risk prediction, with particular attention to the imbalanced nature of the stroke dataset.

The following **six** models are developed and evaluated:

- **Model 1** - **Logistic Regression**: A linear classification model trained using LogisticRegression(max_iter=1000) on the scaled training data.

- **Model 2** - **Balanced Logistic Regression**: A Logistic Regression model using class_weight='balanced' to give greater importance to the minority stroke class.

- **Model 3** - **K-Nearest Neighbors (KNN)**: A distance-based classifier trained using KNeighborsClassifier(n_neighbors=5) on the scaled training data.

- **Model 4 - Decision Tree**: A tree-based classifier trained using DecisionTreeClassifier(random_state=42) on the training data.

- **Model 5** - **Random Forest**: An ensemble classifier consisting of 100 decision trees, implemented using RandomForestClassifier(n_estimators=100, random_state=42).

- **Model 6** - **SMOTE + Logistic Regression**: SMOTE is applied to the training data to balance the stroke classes, followed by Logistic Regression training on the balanced training set.

The models are trained using the training subset and their predictions are generated on the untouched testing subset for subsequent performance evaluation.

Step 11 - Model Evaluation

After training the six machine learning models are evaluated on the unseen testing dataset to measure their predictive performance. Since stroke prediction is a classification problem with an imbalanced target variable, multiple evaluation metrics are considered rather than relying only on accuracy.

The following evaluation measures are used:

- **Accuracy**: Measures the overall proportion of correctly classified patients.
- **Recall**: Measures the proportion of actual stroke cases correctly identified by the model. This is particularly important because missing a stroke case is undesirable in a screening system.
- **Precision**: Measures how many of the patients predicted as having stroke are actually stroke cases.
- **F1-score**: Provides a balance between precision and recall.

- **Confusion Matrix**: Shows the numbers of true negatives, false positives, false negatives and true positives produced by a model.

  The metrics are calculated using the corresponding **sklearn.metrics** functions in the implementation. The evaluation results are subsequently used for **model comparison and selection**.

Step 12 - Model Comparison

The performance of the six trained machine learning models is compared using Accuracy, Recall, Precision and F1-score. The comparison helps identify the strengths and limitations of the different models, particularly with respect to detecting the minority stroke class.

The comparison is performed using the following results obtained from the trained models:

Table: Model Performance Comparison

Model	Accuracy	Recall	Precision	F1 - score
Logistic Regression	**0.952**	**0.020**	**1.000**	**0.039**
KNN	**0.948**	**0.020**	**0.200**	**0.036**
Decision Tree	**0.911**	**0.020**	**0.164**	**0.180**
Random Forest	**0.949**	**0.020**	**0.250**	**0.037**
SMOTE + Logistic Regression	**0.932**	**0.380**	**0.333**	**0.355**
Balanced Logistic Regression	**0.746**	**0.800**	**0.138**	**0.235**

The results show that the standard Logistic Regression model achieves the highest accuracy (**0.952**) but its recall for stroke cases is very low (**0.020**). In contrast, **Balanced Logistic Regression achieves the highest recall of 0.800**, indicating that it identifies a larger

proportion of actual stroke cases. **SMOTE + Logistic Regression achieves the highest F1-score (0.355)** among the six models providing a better balance between recall and precision. These results motivate the subsequent **hyperparameter tuning and final model selection** process.

Step 13 – Hyperparameter Tuning

After comparing the models**, hyperparameter tuning is performed for the Balanced Logistic Regression model** to improve its predictive performance. Hyperparameters are values that control the behaviour of a machine learning model and are selected before the training process.
In this project **GridSearchCV** is used to identify the best value of the regularization parameter **C** for the Balanced Logistic Regression model. The model uses **5 fold cross-validation** and **F1-score** is used as the scoring metric because **both precision and recall are important for the imbalanced stroke-risk dataset.**

The tuning process searches through the specified values of C and selects the combination that produces the best cross-validation F1-score.

The tuning results obtained from the implementation are:

Parameter	Selected Value
Best C	0.1
Best Cross-Validation F1-score	0.228

The tuned Balanced Logistic Regression model is then evaluated on the testing dataset. The resulting performance is:

Metric	Score
Accuracy	0.746
Recall	0.800
Precision	0.138
F1-score	0.236

The tuned model is subsequently considered for final model selection based on its ability to identify stroke cases particularly its recall performance.

Step 14 - Final Model Selection

Based on the model comparison and hyperparameter tuning results, **Tuned Balanced Logistic Regression** is selected as the final prediction model for the MedInsight Stroke-Risk Screening System.
The selection is primarily based on its ability to identify stroke cases. The tuned model achieves a **recall of 0.800** meaning that it correctly identifies a large proportion of the actual stroke cases in the testing dataset. This is particularly important for a stroke-risk screening system, where reducing missed stroke cases is a key consideration.

The selected model uses the best hyperparameter value identified during tuning:

- **Model:** Tuned Balanced Logistic Regression
- **Best C:** 0.1
- **Recall:** 0.800
- **F1-score:** 0.236

The selected model is subsequently saved using **Joblib** and used as the prediction model in the Streamlit application.

Step 15 - Model Saving using Joblib

After selecting the **Tuned Balanced Logistic Regression** model as the final prediction model, the trained model is saved using the **Joblib** library. Saving the trained model allows it to be loaded later for making predictions through the Streamlit application without retraining the model each time.

The selected trained model is saved as a **Joblib file** and can subsequently be loaded during the prediction stage.
This step ensures that the trained model is available for **deployment and future stroke-risk predictions**.

Step 16 - User Authentication and Login

The MedInsight Stroke-Risk Screening System includes a **user authentication mechanism** to control access to the main screening application. Before accessing the prediction interface the user is required to provide valid login credentials through the Streamlit interface.

After successful authentication the user is granted access to the main MedInsight screening system, where patient information can be entered for stroke-risk prediction.

The login mechanism therefore provides an initial access-control layer before the user proceeds to the **patient input and prediction workflow**.

Step 17 - Streamlit Prediction Workflow

After the final model is saved, it is integrated into the **Streamlit-based MedInsight application** for making predictions on new patient data. The application provides an interactive interface through which the user enters the required **demographic, medical and lifestyle information**.

The prediction workflow consists of the following stages:

1. **User Input:** Patient information is entered through the Streamlit interface.
2. **Input Preprocessing:** The entered values are validated, encoded and formatted to match the feature structure used during model training.
3. **Load Saved Model:** The trained Tuned Balanced Logistic Regression model is loaded for prediction.
4. **Prediction:** The preprocessed patient information is passed to the trained model to obtain the predicted stroke outcome.

This workflow allows the trained machine learning model to be used for **new patient predictions without retraining the model**.

Step 18 - Risk Assessment and Prediction Probability

After the prediction is generated, the system calculates the **estimated stroke probability** using the **predict_proba**() function of the selected Tuned Balanced Logistic Regression model. The probability represents the estimated likelihood of the patient belonging to the **stroke class (1)**.
The system then categorizes the estimated probability into three screening risk levels according to the criteria implemented in the Streamlit application:

Stroke Probability	Risk Level
< 20%	**Lower Risk**
20% - 49.99%	**Moderate Risk**
≥ 50%	**Higher Risk**

The probability of the stroke class is obtained using:

```
probability = best_balanced_lr.predict_proba(patient_data)[0][1] #estimate stroke probability
```

The corresponding risk level is determined using the following logic:

```
if probability >= 0.50:
    st.error("🔴 Higher Stroke Risk Detected")
elif probability >= 0.20:
    st.warning("🟡 Moderate Stroke Risk Detected")
else:
    st.success("🟢 Lower Stroke Risk Detected")
```

Thus the system presents the **predicted risk level together with the estimated stroke probability** through the Streamlit interface. This information is subsequently used to provide personalised screening guidance.

Step 19 - Personalised Screening Guidance

After determining the predicted stroke outcome, estimated probability and corresponding risk level, the MedInsight system provides **personalised screening guidance** through the Streamlit interface. The guidance is presented according to the prediction result and relevant patient risk factors.

For patients predicted as **No Stroke** the system provides **preventive guidance** aimed at maintaining healthy lifestyle practices and reducing potential risk factors.

For patients predicted as **Stroke** the system provides guidance based on the identified **risk factors and preventive measures**, encouraging appropriate attention to the screening result. The personalised guidance is displayed together with the prediction and risk assessment, making the output more informative and useful for the user.

Step 20 - Model Explainability & Analytics

To improve the interpretability and analytical understanding of the stroke-risk prediction system additional explainability and evaluation techniques are applied to the selected Tuned Balanced Logistic Regression model. These techniques help examine the factors influencing predictions and assess the model's behaviour beyond basic classification metrics.

The following analytical methods are incorporated:

- **SHAP Analysis**: SHAP (**SHapley Additive exPlanations**) is used to examine how individual features contribute to the model's predictions. A LinearExplainer is created for the Tuned Balanced Logistic Regression model, and individual patient predictions can be explained using SHAP values.

- **Feature Importance**: SHAP-based analysis is used to identify which input features have greater influence on the model's predictions.

- **Correlation Analysis**: A correlation heatmap is generated for the numerical attributes to examine relationships among the input variables and the target variable.

- **ROC-AUC Analysis**: The ROC curve and Area Under the Curve (AUC) are calculated using the predicted stroke probabilities to evaluate the model's ability to distinguish between stroke and no-stroke cases.

- **Recall-Focused Analysis**: Different probability thresholds are tested to observe their effect on recall, precision and F1-score. This analysis is particularly relevant because identifying actual stroke cases is an important objective of the screening system.

These analyses provide additional insight into **the model behaviour, influential features and classification performance** and support the interpretation of the final prediction system.

CHAPTER 6 – ALGORITHMS AND TECHNIQUES

6.1 Supervised Machine Learning

Supervised Machine Learning is a type of machine learning in which a model is trained using a dataset containing known input features and corresponding target labels. The model learns the relationship between the input variables and the target variable and uses the learned patterns to make predictions on unseen data.

In the **MedInsight Stroke-Risk Screening System**, supervised learning is used to predict whether a patient belongs to the **No Stroke (0)** or **Stroke (1)** class. The input features include demographic, medical and lifestyle attributes such as age, hypertension, heart disease, average glucose level, BMI and encoded categorical attributes.

The project trains and compares multiple supervised classification models, including:

- **Logistic Regression**
- **K-Nearest Neighbours (KNN)**
- **Decision Tree**
- **Random Forest**
- **SMOTE + Logistic Regression**
- **Balanced Logistic Regression**

These models learn from the prepared training dataset and generate predictions for the unseen testing dataset. Their performance is then compared using classification evaluation metrics to identify the most suitable model for stroke-risk screening.

6.2 Binary Classification

Binary Classification is a supervised machine learning task in which the target variable contains only **two possible classes**. The model learns to distinguish between these two classes based on the available input features.

In the **MedInsight Stroke-Risk Screening System**, the target variable is the **stroke** attribute, which represents the patient's stroke outcome:

Target Value	Meaning
0	**No Stroke**
1	**Stroke**

The machine learning models analyse the patient's demographic, medical and lifestyle features and classify the patient into one of these two categories.

Binary classification is suitable for this project because the primary objective is to determine whether a patient is **predicted to have a stroke or not**, based on the patterns learned from the training data.

The binary prediction output is subsequently used by the application to calculate the **stroke probability** and assign the corresponding **Lower, Moderate or Higher Risk** level.

6.3 Logistic Regression

Logistic Regression is a supervised machine learning algorithm used primarily for classification problems. It estimates the probability that an input sample belongs to a particular class and then assigns the sample to the appropriate class based on the predicted probability.
For binary classification Logistic Regression uses the **sigmoid function** to convert the model's output into a probability between **0 and 1**:

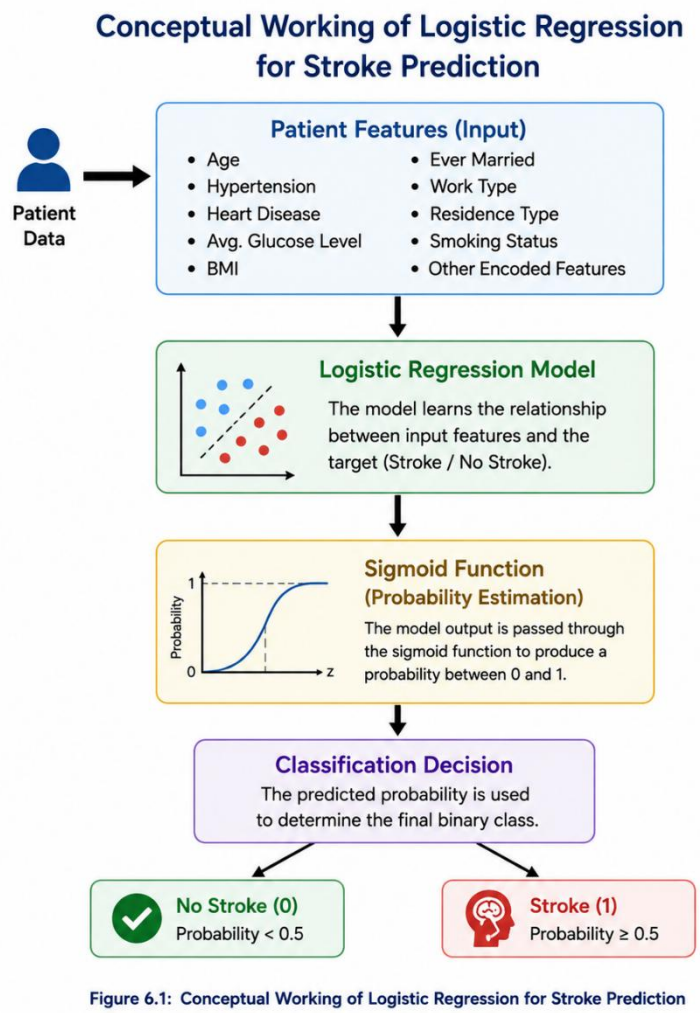


Figure 6.1: Conceptual Working of Logistic Regression for Stroke Prediction

Application in the MedInsight System

In the **MedInsight Stroke-Risk Screening System**, Logistic Regression is trained to perform binary classification of patients into:

- **0 - No Stroke**
- **1 - Stroke**

The model uses the preprocessed demographic, medical and lifestyle features as input. The features are first transformed through the preprocessing pipeline and scaled using **StandardScaler** before being provided to the Logistic Regression model.

The project uses Logistic Regression in multiple forms during model comparison:

1. **Standard Logistic Regression**
2. **SMOTE + Logistic Regression**, where SMOTE is used to address class imbalance before training.
3. **Balanced Logistic Regression**, which incorporates class balancing during model training.

The Balanced Logistic Regression model is further tuned using **GridSearchCV** to identify the best value of the regularization parameter **C**. The best value obtained during tuning is **C = 0.1** and the resulting tuned Balanced Logistic Regression model is selected as the final model for the prediction system.

Prediction Probability

One important advantage of Logistic Regression in this project is that it can provide a **class probability**. The probability of the stroke class is obtained using predict_proba() in the Streamlit application:

```
probability = best_balanced_lr.predict_proba(patient_data)[0][1] #estimate stroke probability
```

The obtained probability is subsequently used by the MedInsight application for **risk assessment**, where the patient is categorized as Lower, Moderate or Higher Risk according to the defined probability thresholds.

Advantages: Logistic Regression is simple, computationally efficient, suitable for binary classification and provides probability estimates that can be used for stroke-risk assessment.

Limitations: Logistic Regression may not capture complex non-linear relationships between features as effectively as tree based algorithms and can be affected by class imbalance if balancing techniques are not applied.

6.4 K-Nearest Neighbours (KNN)

K-Nearest Neighbours (KNN) is a supervised machine learning algorithm used for classification. It classifies a new data point based on the classes of its nearest neighbouring data points in the feature space.

In KNN the value of **K** represents the number of neighbouring observations considered for making a prediction. The class occurring most frequently among the selected neighbours is assigned to the new observation.

Application in the MedInsight System

In the **MedInsight Stroke-Risk Screening System** KNN is used as one of the classification models for comparison with the other machine learning algorithms.

The KNN model is trained using the **scaled training features** and their corresponding stroke labels. Feature scaling is important for KNN because the algorithm calculates distances between observations and features with larger numerical ranges could otherwise have greater influence on the distance calculation.

The trained KNN model is evaluated on the unseen testing dataset using **Accuracy, Recall, Precision and F1-score**. Its performance is then compared with the other models to determine the most suitable model for the project.

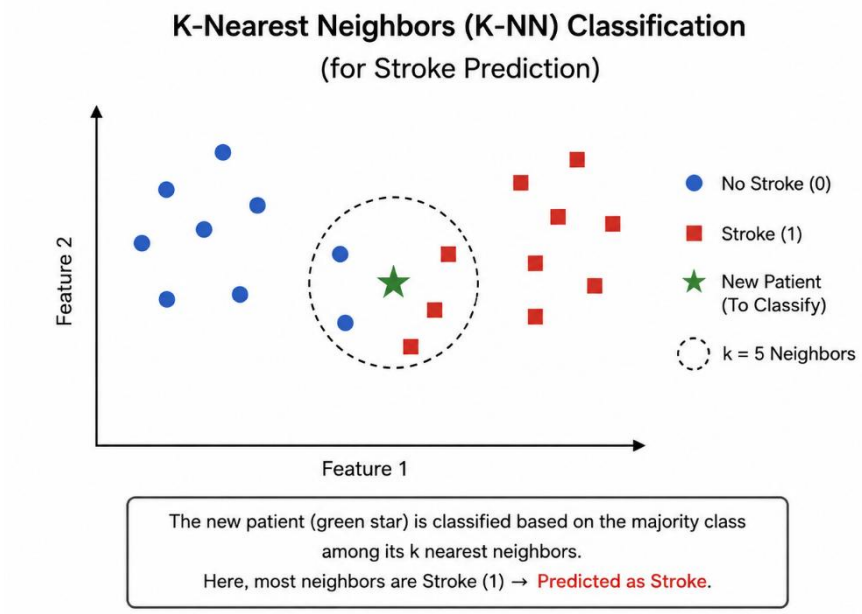


Figure 6.2: Simplified Conceptual Illustration of K-Nearest Neighbors for Stroke Classification

Advantages: KNN is simple and easy to understand, can handle non-linear relationships between features, and does not require a complex model-building process.

Limitations: KNN can become computationally expensive when making predictions on larger datasets, and its performance depends on the choice of **K** and the distance between observations. It is also sensitive to feature scaling and irrelevant features.

6.5 Decision Tree

Decision Tree is a supervised machine learning algorithm that performs classification by dividing the dataset into a series of decision rules. Each internal node represents a decision based on a feature, each branch represents the outcome of that decision and the leaf nodes represent the final predicted class.

Application in the MedInsight System

In the **MedInsight Stroke-Risk Screening System** Decision Tree is used as one of the classification models for comparing different machine learning approaches. The model learns decision rules from the training data using the prepared patient features and corresponding stroke labels.

The trained Decision Tree is then applied to the unseen testing dataset to predict whether a patient belongs to the **No Stroke (0)** or **Stroke (1)** class. Its performance is evaluated using **Accuracy, Recall, Precision and F1-score** and compared with the other models.

Advantages: Decision Tree is easy to understand and interpret, can capture non-linear relationships between features and does not require extensive mathematical assumptions about the data.

Limitations: Decision Tree models can become overly complex and may suffer from overfitting. Their performance can also be sensitive to small changes in the training data.

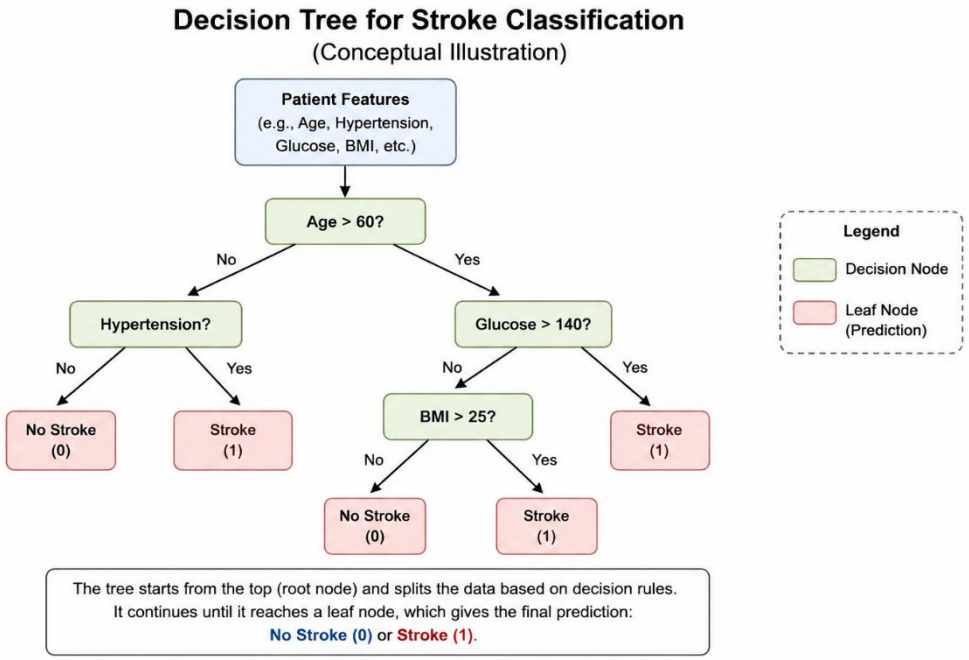


Figure 6.3: Conceptual Working of Decision Tree for Stroke Classification

6.6 Random Forest

Random Forest is a supervised ensemble learning algorithm that combines multiple Decision Trees to improve classification performance. The individual trees are trained using different subsets of the data and their predictions are combined to obtain the final class.

Application in the MedInsight System

In the **MedInsight Stroke-Risk Screening System** Random Forest is used as one of the classification models for comparison. It predicts whether a patient belongs to **No Stroke (0)** or **Stroke (1)** using the prepared patient features.

Advantages: Random Forest can capture non-linear relationships, provides good predictive performance, and is less prone to overfitting than a single Decision Tree.

Limitations: It requires more computational resources than a single Decision Tree and is comparatively less interpretable because it combines multiple trees.

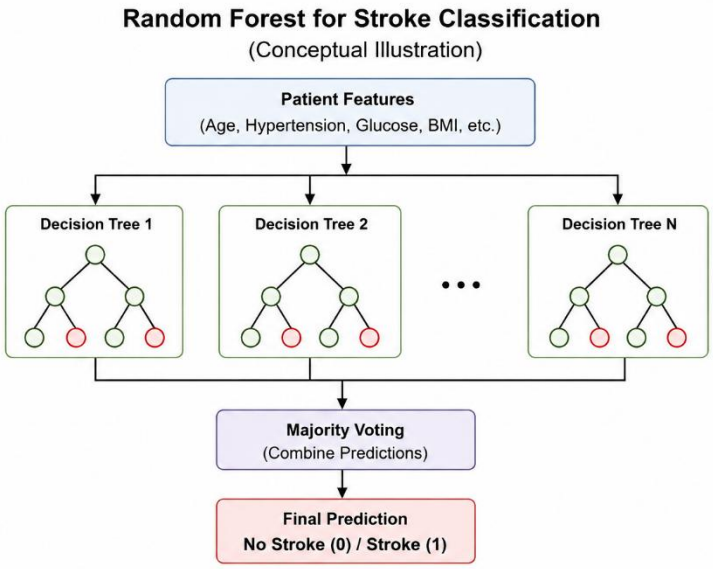


Figure 6.4: Conceptual Working of Random Forest for Stroke Classification

6.7 SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE is a technique used to address **class imbalance** in a dataset. It generates new synthetic samples for the minority class instead of simply duplicating existing observations.

Application in the MedInsight System

In the **MedInsight Stroke-Risk Screening System** SMOTE is applied to the training data to increase the representation of the minority **Stroke (1)** class. This helps the model learn patterns associated with stroke cases more effectively.

The project uses **SMOTE + Logistic Regression** as one of the models during model comparison. The testing data remains untouched so that the model can be evaluated fairly on unseen data.

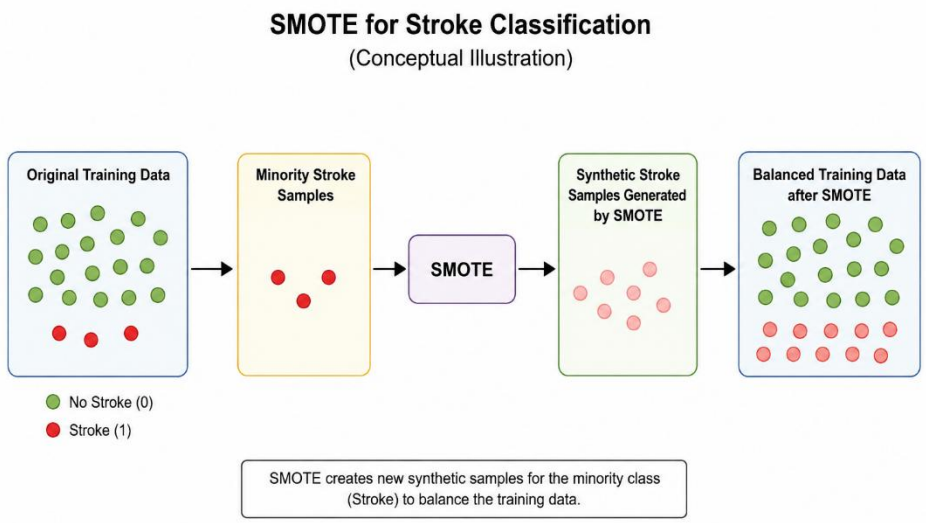


Figure 6.5: Conceptual Working of SMOTE for Stroke Classification

Advantages: SMOTE helps reduce the effect of class imbalance, improves the representation of minority-class samples and can help classification models identify minority cases more effectively.

Limitations: SMOTE may generate synthetic samples that do not perfectly represent real patients and can potentially introduce overlapping or noisy observations if applied inappropriately.

6.8 Balanced Logistic Regression

Balanced Logistic Regression is a modified form of Logistic Regression designed to handle **imbalanced classification datasets**. It gives greater importance to the minority class during model training so that the model does not become overly biased toward the majority class.

Application in the MedInsight System

In the **MedInsight Stroke-Risk Screening System** Balanced Logistic Regression is used to improve the identification of the minority **Stroke (1)** class. The model is trained using the prepared and scaled patient features with class balancing enabled.

It is included in the model comparison along with Standard Logistic Regression, KNN, Decision Tree, Random Forest and SMOTE + Logistic Regression. The Balanced Logistic Regression model achieves **0.800 recall** in the testing evaluation and is subsequently selected for further hyperparameter tuning.

The regularization parameter **C** is then optimized using **GridSearchCV**, with **C = 0.1** selected as the best value.

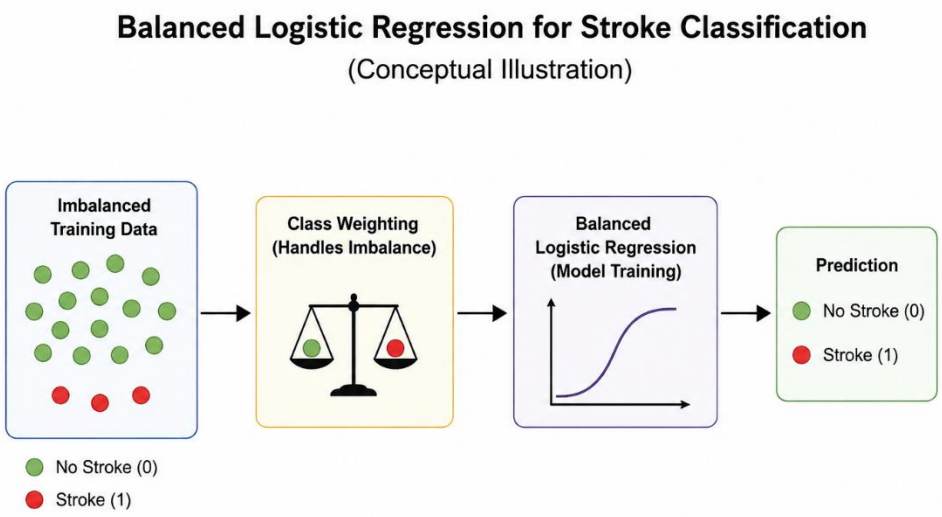


Figure 6.6: Conceptual Working of Balanced Logistic Regression for Stroke Classification

Advantages: Balanced Logistic Regression gives greater importance to minority-class cases, making it more suitable for imbalanced stroke prediction and helping improve the identification of stroke cases.

Limitations: It may produce lower precision when the model prioritizes identifying more minority class cases and it may not capture complex non-linear relationships as effectively as tree-based models.

6.9 Label Encoding

Label Encoding is a preprocessing technique used to convert categorical values into numerical values so that machine learning algorithms can process them.

Application in the MedInsight System

In the MedInsight Stroke-Risk Screening System, **Label Encoding is applied to the Gender feature** to convert its categorical values into numerical labels. The encoded feature is then used along with the other processed numerical and categorical features for model training and prediction.

Advantages: Label Encoding is simple, computationally efficient and converts categorical information into a numerical format that can be processed by machine learning algorithms.

Limitations: For nominal categories assigning numerical labels may introduce an artificial ordering between categories even when no such relationship exists.

Table: Example of Label Encoding

Original Feature	Original Categories	Label Encoded Values
Gender	Female, Male, Other	0, 1, 2

6.10 Train-Test Split

Train-Test Split is a data-splitting technique used to divide a dataset into separate training and testing sets. The training set is used to train the machine learning model, while the testing set is used to evaluate its performance on unseen data.

Application in the MedInsight System

In the MedInsight Stroke-Risk Screening System, the prepared dataset is divided into training and testing sets before model training. The training data is used to learn patterns between the patient features and the stroke outcome, while the testing data is kept separate for final evaluation.

The test data is not used for model training, helping provide a fair estimate of the model's performance on new patient data.

Advantages: Train-test splitting helps evaluate how well a model generalizes to unseen data and reduces the possibility of evaluating the model only on data it has already seen.

Limitations: The model's performance can vary depending on how the data is split particularly when the dataset is relatively small or imbalanced.

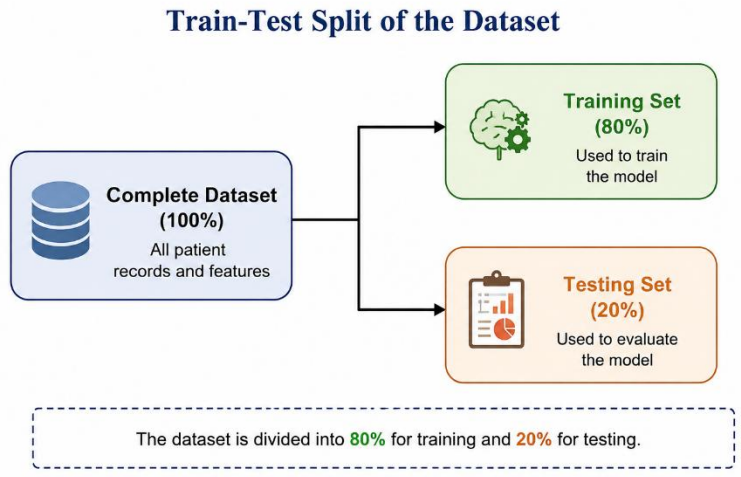


Figure 6.7: Train-Test Split of the Dataset

6.11 Feature Scaling using StandardScaler

Feature Scaling is a preprocessing technique used to bring numerical features to a comparable scale. In the **MedInsight Stroke-Risk Screening System**, StandardScaler is used to standardize the numerical features before they are provided to the machine learning models.
StandardScaler transforms each feature using its **mean and standard deviation**, producing standardized values with a mean close to 0 and a standard deviation close to 1. This is particularly useful for models such as **Logistic Regression and KNN**, where the scale of input features can affect the model's performance.

Application in the MedInsight System

After preprocessing and splitting the dataset, the scaler is **fitted on the training data** and then used to transform both the training and testing data. This prevents information from the testing set from influencing the scaling process.

The scaled features are subsequently used for model training and evaluation.

Advantages: StandardScaler puts numerical features on a comparable scale, which can improve the performance of scale-sensitive algorithms and make the optimization process more stable.

Limitations: StandardScaler can be affected by extreme outliers because the mean and standard deviation are used during standardization.

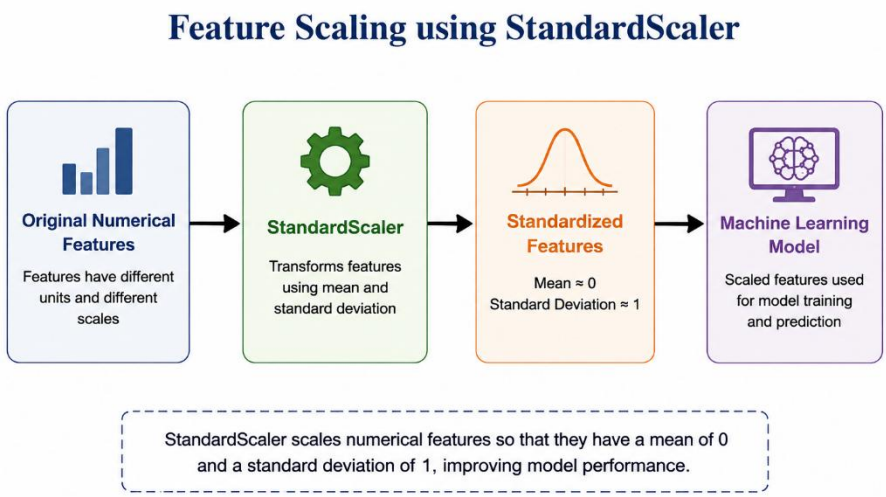


Figure 6.8: Feature Scaling using StandardScaler

6.12 Evaluation Metrics

- **Accuracy:** Accuracy measures the overall correctness of the prediction model. It is calculated as the ratio of correctly predicted patient records to the total number of predictions made.

Formula: Accuracy = (Number of Correct Predictions) / (Total Number of Predictions)

- **Precision:** Precision measures how many patients predicted as having stroke actually belong to the Stroke class. A high precision value indicates that the model produces fewer false-positive predictions.

Formula (per class): Precision = True Positives / (True Positives + False Positives)

- **Recall:** Recall measures how many actual stroke cases are correctly identified by the model. It is particularly important in stroke-risk screening because missing an actual stroke case can be undesirable.

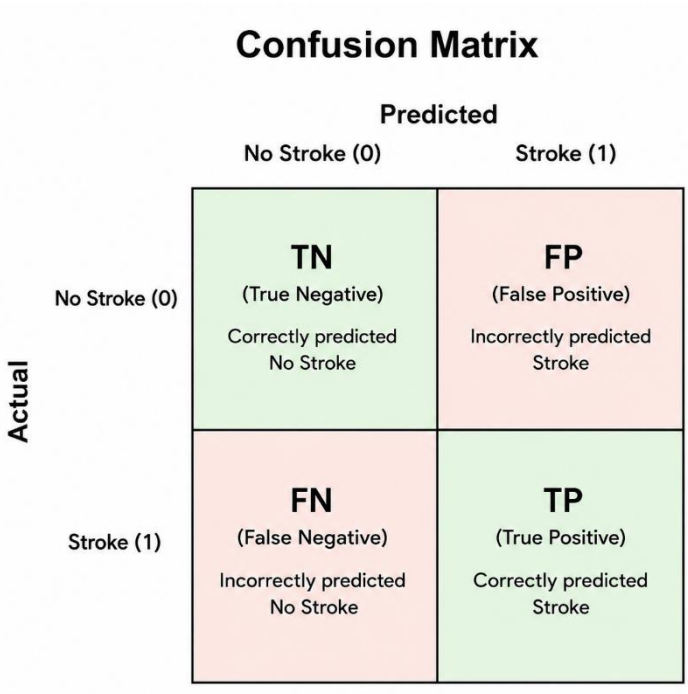
Formula (per class): Recall = True Positives / (True Positives + False Negatives)

- **F1-score:** F1-score provides a balance between Precision and Recall and is particularly useful when the dataset contains class imbalance.

Formula: F1-score = 2 × (Precision × Recall) / (Precision + Recall)

- **Confusion Matrix:** A **Confusion Matrix** is a classification evaluation technique used to compare the actual outcomes with the predicted outcomes of a machine learning model. It consists of four components: **True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN)**.

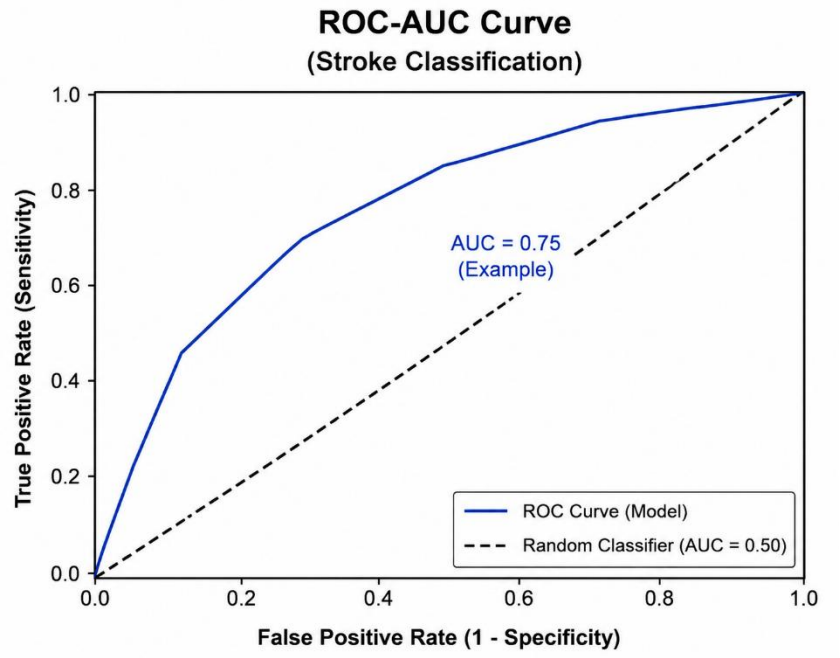
  For the MedInsight Stroke-Risk Screening System the confusion matrix helps identify correctly and incorrectly classified **Stroke (1)** and **No Stroke (0)** cases. It is particularly useful for understanding the model's **Precision and Recall**.



- **ROC-AUC:** The **Receiver Operating Characteristic (ROC)** curve is used to evaluate the classification performance of a model at different probability thresholds. It plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR).**

  The **Area Under the Curve (AUC**) summarizes the overall ability of the model to distinguish between **Stroke (1) and No Stroke (0)** cases. A higher AUC generally indicates better class discrimination.

  In the MedInsight system ROC-AUC is calculated using the predicted stroke probabilities produced by the final model.



Example

Suppose the test dataset contains **40 patient records**, of which **20 patients actually have stroke**. If the model correctly identifies **18 stroke patients** but incorrectly predicts **3 No Stroke patients as having stroke**, then:

Accuracy = (Correct Predictions / Total Predictions)

Precision = 18 / (18 + 3) = 85.7%

Recall = 18 / 20 = 90%

This gives:

- **TP = 18**
- **FP = 3**
- **FN = 2**
- **TN = 17**

Therefore, Accuracy = (18 + 17) / 40 = 87.5%

6.13 Hyperparameter Tuning using GridSearchCV

Hyperparameter tuning is the process of selecting the best values of a model's hyperparameters to improve its performance. In the MedInsight system **GridSearchCV** is used to systematically test different parameter values and identify the best configuration for the Balanced Logistic Regression model.

Application in the MedInsight System

The project uses **GridSearchCV** to tune the regularization parameter **C** of the Balanced Logistic Regression model. Different values of C are evaluated using cross-validation and the parameter combination giving the best performance is selected.

In the final model **C = 0.1** is selected as the best value and the resulting tuned Balanced Logistic Regression model is used for the prediction system.

Advantages: GridSearchCV provides a systematic way to search for suitable hyperparameter values and can improve model performance compared with using default parameters.

Limitations: Grid search can require considerable computational time because multiple parameter combinations and cross-validation folds are evaluated.

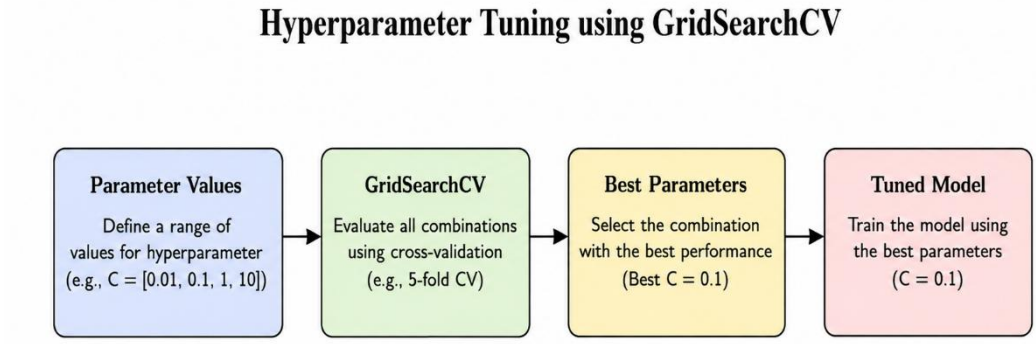


Figure 6.11: Hyperparameter Tuning Process using GridSearchCV

6.14 SHAP Explainability

SHAP (SHapley Additive exPlanations) is an explainability technique used to understand how individual features influence a machine learning model's prediction.

Application in the MedInsight System

In the MedInsight Stroke-Risk Screening System SHAP is applied to the **Tuned Balanced Logistic Regression model** to explain individual stroke-risk predictions.

shap.LinearExplainer is used with the trained model and training data.

The SHAP values indicate whether a feature **increases or decreases the model output** for a particular patient. The system also identifies the **Top 5 factors influencing the prediction** and displays them through a SHAP waterfall chart.

Advantages: It makes model predictions easier to understand, shows the contribution of individual features and improves transparency of the prediction process.

Limitation: SHAP explanations describe the model's learned relationships and **do not establish that a feature causally produces stroke**.

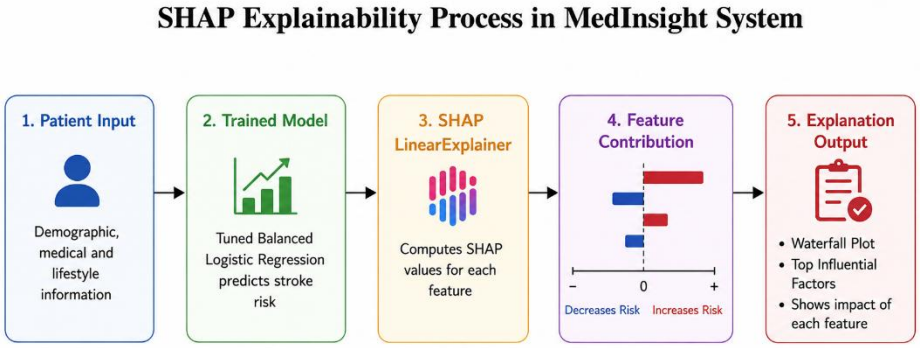


Figure 6.12: SHAP Explainability Process in the MedInsight System

CHAPTER 7 – IMPLEMENTATION

7.1 Tools and Technologies

The implementation of the MedInsight: Stroke Risk Assessment and Explainable Prediction System was carried out using a combination of programming languages, data-processing libraries, machine learning frameworks, explainability tools, visualization libraries, and deployment technologies. These tools were selected to support the complete workflow of the system, including data preprocessing, machine learning model development, model evaluation, hyperparameter tuning, explainable prediction, web-based interaction, and report generation.

The major tools and technologies used in the implementation are described below.

Technology	Application in the System
Python	Used as the primary programming language for implementing the complete system.
Pandas	Used for loading, inspecting, cleaning and processing the dataset.
NumPy	Used for numerical and array-based operations.
Scikit-learn	Used for preprocessing, machine learning models, evaluation and hyperparameter tuning.
Imbalanced-learn	Used to apply SMOTE for handling class imbalance.
SHAP	Used to explain model predictions and identify important feature contributions.
Matplotlib & Seaborn	Used to generate graphs and visualizations for data and model analysis.
Joblib	Used to save and load the trained machine learning model.
Streamlit	Used to develop the interactive MedInsight web application.
Reportlab	Used to generate downloadable PDF assessment reports.
Pillow (PIL)	Used in image handling in the application

7.2 Hardware Requirements

The MedInsight system uses conventional machine learning algorithms and does not require specialized hardware such as a GPU. It can be developed and executed on a standard computer with the following minimum requirements:

Component	Requirement
Processor	Intel Core i3 / AMD equivalent or higher
RAM	4 GB minimum, 8 GB recommended
Storage	At least 1 GB free space
GPU	Not required; CPU is sufficient
Display	Standard monitor/display
Internet	Required for accessing the web-based Streamlit application

7.3 Software Requirements

The following software and development components are required to implement and run the MedInsight system:

Software / Component	Requirement / Use
Operating System	Windows / Linux / macOS
Python	3.13 (64-bit)
IDE	Visual Studio Code
Libraries	Pandas, NumPy, Scikit-learn, Imbalanced-learn, SHAP, Matplotlib, Seaborn, Joblib, ReportLab,Pillow(PIL)
Web Framework	Streamlit
Web Browser	Chrome, Edge or any modern browser

7.4 Development Environment

The MedInsight system was developed using **Python 3.13 (64-bit)** in **Visual Studio Code** on a Windows operating system. The development environment was used for dataset preprocessing, machine learning model training and evaluation, hyperparameter tuning, SHAP-based explainability and development of the Streamlit application.

The trained model was saved using **Joblib** and integrated with the Streamlit application for real-time patient prediction, risk assessment, explainability and PDF report generation.

7.5 Coding Approach

The MedInsight system was implemented through a structured coding approach covering **data preprocessing, model development, model evaluation, final model selection and Streamlit application development**. The implementation was divided into modules so that each stage of the system could be developed and tested separately.

The coding approach is divided into **3** major parts:

A. Model Development and Training

Modules 1–12 cover dataset preparation, preprocessing, model training, evaluation, hyperparameter tuning, and saving the final model.

B. Streamlit Application
Modules 13–18 cover the user interface, patient input, prediction, risk assessment, SHAP explanation, guidance, and PDF report generation.

C. Model Analytics
Module 19 covers the model performance and analytical visualizations.

Module 1 - Importing Libraries

Purpose: To import the required Python libraries for data preprocessing, machine learning, model evaluation, visualization, SHAP explainability, model saving, Streamlit development and PDF report generation.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import recall_score
from sklearn.metrics import precision_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV
from imblearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score
import shap
from sklearn.metrics import roc_auc_score, roc_curve
import seaborn as sns
import numpy as np
from sklearn.metrics import confusion_matrix
```

Figure 7.1: Importing Libraries for Machine Learning Model Development

Explanation: The required Python libraries and modules are imported for different stages of the MedInsight machine learning workflow. **Pandas** is used for data handling while **Scikit-learn** provides LabelEncoder, train_test_split, StandardScaler, the classification algorithms, evaluation metrics and GridSearchCV for hyperparameter tuning. **KNN, Decision Tree and Random Forest** are imported for model comparison. **SMOTE** from Imbalanced-learn is used to address class imbalance, while **SHAP** is used for model explainability. **Seaborn** supports visualization and the imported ROC-AUC and confusion matrix functions are used for model evaluation.

Module 2 - Dataset Loading and Initial Inspection

Purpose: To load the stroke dataset and inspect its basic structure before applying preprocessing and model development.

```
df = pd.read_csv('MedInsight/Stroke_prediction.csv')

print(df.head())
print(df.shape)
print(df.info())

print(df.isnull().sum())
print(df['bmi'].describe())
print(df['stroke'].value_counts())
```

Figure 7.2: Loading and Initial Inspection of the Stroke Dataset

Explanation: The stroke dataset is loaded into a Pandas DataFrame using read_csv(). The initial inspection checks the first few records, dataset dimensions, column information, missing values, BMI statistics, and the distribution of the target variable (stroke). This helps identify the dataset characteristics and preprocessing requirements before model development.

Module 3 - Missing Value Handling

Purpose: To identify and handle missing values in the dataset before applying further preprocessing and model training.

```
print(df.isnull().sum())
print(df['bmi'].describe())
df['bmi'] = df['bmi'].fillna(df['bmi'].median())
print(df.isnull().sum())
```

Figure 7.3: Missing Value Handling in the Stroke Dataset

Explanation: The dataset is first checked for missing values using isnull().sum(). The bmi feature contains missing values, which are handled by replacing them with the **median BMI value**. The dataset is then checked again to confirm that the missing values have been addressed.

Module 4 - Categorical Encoding and Feature Preparation

Purpose: To convert categorical features into numerical form and prepare the dataset for machine learning.

```
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])
print(df['gender'].unique())

df['ever_married'] = df['ever_married'].map({'No': 0 , 'Yes': 1})
print(df['ever_married'].unique())
df['Residence_type']= df['Residence_type'].map({'Rural':0, 'Urban': 1})
print(df['Residence_type'].unique())

#one hot
df = pd.get_dummies(df, columns = ['work_type'] , dtype = int)
print(df.head())
df = pd.get_dummies(df , columns = ['smoking_status'] , dtype = int)
print(df.head())
print(df.info())
#remove patient identifier as not a predictive feature
df = df.drop('id' , axis = 1)
print(df.head())
print(df.shape)
```

Figure 7.4: Categorical Encoding and Feature Preparation

Explanation: Label Encoding is applied to gender, while ever_married and Residence_type are converted using numerical mappings. One-hot encoding is applied to work_type and smoking_status to create separate numerical features. The id column is then removed because it is only a patient identifier and is not used as a predictive feature.

Module 5 - Feature and Target Separation

Purpose: To separate the input features from the target variable so that the machine learning models can learn to predict the stroke outcome**.**

```
# seperate inputs and target
X = df.drop('stroke' , axis=1)
y = df['stroke']
print(X.shape)
print(y.shape)
```

Figure 7.5: Separation of Features and Target Variable

Explanation: After preprocessing the stroke column is separated as the target variable y, while all remaining predictive attributes are stored in X. The shapes of both objects are checked to confirm that the separation has been performed correctly.

Module 6 - Train-Test Split

Purpose: To divide the dataset into separate training and testing sets so that the models can be trained on one portion of the data and evaluated on unseen data.

```
#train - test - split

X_train , X_test , y_train , y_test = train_test_split(X,y, test_size = 0.2 , random_state = 42, stratify = y)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

Figure 7.6: Train-Test Split Implementation

Explanation: The dataset is divided into 80% training data and 20% testing data using train_test_split(). A fixed random_state ensures reproducible splitting, while stratify=y maintains a similar proportion of Stroke and No Stroke cases in both sets.

Module 7 - Feature Scaling

Purpose: To standardize the numerical feature values so that features with different scales can be used effectively during model training.

```
#scaling

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Figure 7.7: Feature Scaling using StandardScaler

Explanation: StandardScaler is used to fit the scaling transformation on the training data and then apply the same transformation to both training and testing data. This prevents information from the test set from influencing the scaling process and ensures that both datasets are transformed consistently.

Module 8 - Model Training

Purpose: To train the selected machine learning algorithms using the prepared training data and build models for stroke prediction.

```
#Create LR model
model_lr = LogisticRegression(max_iter = 1000)
#train model
model_lr.fit(X_train_scaled , y_train)
#max_iter=1000 simply gives the model enough iterations to converge.

#Predict
y_pred_lr = model_lr.predict(X_test_scaled)
print(y_pred_lr[:10])
```

Figure 7.8: Logistic Regression Model Training and Prediction

The same training and prediction procedure was applied to KNN, Decision Tree, Random Forest and the balanced Logistic Regression model.

Explanation: Six machine learning approaches are implemented and compared in the project: **Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression**. Scaled data is used for Logistic Regression and KNN, while tree-based models are trained using the unscaled feature data. Each model is fitted using the training dataset and then used to generate predictions on the test dataset.

Module 9 - Handling Class Imbalance using SMOTE

Purpose: To address the imbalance between the Stroke and No Stroke classes in the training data by generating synthetic samples for the minority class.

```
#SMOTE
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(
    X_train,
    y_train
)
print(y_train_smote.value_counts())
```

Figure 7.9: SMOTE-Based Class Balancing of the Training Data

Explanation: SMOTE (Synthetic Minority Over-sampling Technique) is applied **only to the training data** to increase the representation of the minority Stroke class. The original test data remains unchanged so that model performance can be evaluated on unseen, real data. The balanced training data is then used to train the SMOTE + Logistic Regression model, and its performance is compared with the other evaluated models.

Module 10 - Model Evaluation and Comparison

Purpose: To evaluate the performance of the implemented machine learning models using standard classification metrics and compare their results.



```
#Evaluate
accuracy_lr = accuracy_score(y_test , y_pred_lr)
print("Logistic Regression Accuracy:",accuracy_lr)

recall_lr = recall_score(y_test ,  y_pred_lr)
print("Logistic Regression Recall:", recall_lr)

precision_lr = precision_score(y_test , y_pred_lr)
print("Logistic Regression Precision" , precision_lr)

f1_lr = f1_score(y_test , y_pred_lr)
print("Logistic Regression F1: ", f1_lr)
 comparison = pd.DataFrame({
     "Model": [
         "Logistic Regression",
         "KNN",
         "Decision Tree",
         "Random Forest",
         "SMOTE + Logistic Regression",
         "Balanced Logistic Regression"
     ],

     "Accuracy": [
         accuracy_lr,
         accuracy_knn,
         accuracy_dt,
         accuracy_rf,
         accuracy_smote,
         accuracy_lr_balanced
     ],
     "Recall": [
         recall_lr,
         recall_knn,
         recall_dt,
         recall_rf,
         recall_smote,
         recall_lr_balanced
     ],

     "Precision": [
         precision_lr,
         precision_knn,
         precision_dt,
         precision_rf,
         precision_smote,
         precision_lr_balanced
     ],

     "F1": [
         f1_lr,
         f1_knn,
         f1_dt,
         f1_rf,
         f1_smote,
         f1_lr_balanced
     ]
})
print(comparison.round(3))
```

Figure 7.10: Performance Comparison of Machine Learning Models

Explanation: The models are evaluated using **Accuracy, Recall, Precision and F1-score**. Since stroke prediction is an imbalanced classification problem, Recall is particularly important because it measures the model's ability to correctly identify stroke cases. The calculated metrics are then organized into a comparison DataFrame to compare the performance of the different models.

Module 11 – Cross Validation and Hyperparameter Tuning

Purpose: To identify the best hyperparameter settings for Logistic Regression, including the regularization parameter and class-weight configuration, and improve its classification performance.

```
lr_tune = LogisticRegression(max_iter=2000)

# Values we want to test
param_grid_lr = {
    'C': [0.01, 0.1, 1, 10],
    'class_weight': [None, 'balanced']
}
# Try every combination using 5-fold cross-validation
grid_lr = GridSearchCV(
    lr_tune,
    param_grid_lr,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
# Train using the SMOTE-balanced training data
grid_lr.fit(X_train_smote_scaled, y_train_smote)# Best hyperparameter combination found
print("Best Parameters:", grid_lr.best_params_)
# Best average F1-score from 5-fold cross-validation
print("Best Cross-Validation F1:", grid_lr.best_score_)

# Get the best Logistic Regression model
best_lr = grid_lr.best_estimator_
```

Figure 7.11: Hyperparameter Tuning using GridSearchCV

Explanation: GridSearchCV tests different values of the Logistic Regression hyperparameter C and class-weight configuration using **5-fold cross-validation.** The combinations are evaluated using **F1-score**, and the combination producing the best cross-validation performance is selected. The resulting best estimator is then used to make predictions on the unseen test data.

Module 12 - Final Model Selection and Model Saving

Purpose: To select the best-performing Balanced Logistic Regression model after hyperparameter tuning and save it for use in the MedInsight application.

```
# Hyperparameter tuning for Balanced Logistic Regression

lr_balanced_tune = LogisticRegression(
    class_weight='balanced',
    max_iter=2000
)
param_grid_balanced = {
    'C': [0.01, 0.1, 1, 10, 100]
}
grid_balanced = GridSearchCV(
    lr_balanced_tune,
    param_grid_balanced,
    cv=cv,
    scoring='f1',
    n_jobs=-1
)
grid_balanced.fit(X_train, y_train)

print("Best C:", grid_balanced.best_params_)
print("Best CV F1:", grid_balanced.best_score_)

# Best tuned Balanced Logistic Regression
best_balanced_lr = grid_balanced.best_estimator_
import joblib
joblib.dump(best_balanced_lr, "balanced_lr_model.pkl")
# joblib.dump(X_train , "x_train.pkl")
```

Figure 7.12: Final Model Selection and Model Saving

Module 13 - Streamlit User Interface

Purpose: To provide an interactive web-based interface for entering patient information and accessing the MedInsight stroke-risk screening system.

```
st.markdown(
    """<div style="text-align:center;margin:8px 0 34px 0;"><div style="font-size:50px;font-weight:750;letter-spacing:-1.5px;
    line-height:1.1;background:linear-gradient(90deg,#1557A6 0%,#1769B0 30%,#1599A8 65%,#12B8B0 100%);
    -webkit-background-clip:text;-webkit-text-fill-color:transparent;">MedInsight</div><div style="display:flex;
    align-items:center;justify-content:center;margin-top:6px;"><div style="width:125px;height:2px;background:linear-gradient
    (90deg,#1557A6, 50%, #12B8B0 100%);"></div><svg width="55" height="22" viewBox="0 0 55 22" style="margin:0;"><polyline
    points="0,11 15,11 20,11 24,3 28,19 32,11 55,11" fill="none" stroke="#12AEB0" stroke-width="2.2" stroke-linecap="round"
    stroke-linejoin="round"></polyline></svg><div style="width:125px;height:2px;background:linear-gradient(90deg,#12B8B0,
    #1557A6);"></div></div></div>""",
    unsafe_allow_html=True

)

st.write("Explainable Stroke Risk Prediction & Screening Support System")
st.caption(
    "An explainable machine-learning system for stroke-risk screening "
    "and decision support."
)

# Patient inputs
st.subheader("👤 Patient Information")
col1, col2 = st.columns(2)
with col1:
    age = st.number_input("Age", min_value=1, max_value=100, value=50, step = 1)
```

```
    ever_married = st.selectbox(
        "Ever Married",
        ["No" , "Yes"]
    )
    ever_married = 1 if ever_married == "Yes" else 0

with col2:
    gender = st.selectbox(
        "Gender",
        ["Female" , "Male" , "Other"]
    )
    gender = 0 if gender == "Female" else 1 if gender == "Male" else 2


    Residence_type = st.selectbox(
        "Residence Type",
        ["Urban" , "Rural"]
    )
    Residence_type = 1 if Residence_type == "Urban" else 0


st.subheader("❤️ Health Factors")

col1 , col2 = st.columns(2)
with col1:
    hypertension = st.selectbox(
        "Hypertension",
        ["No", "Yes"]
    )
    hypertension = 1 if hypertension == "Yes" else 0
    avg_glucose_level = st.number_input(
        "Average Glucose Level",
        min_value=0.0,
        max_value=300.0,
        value = 100.0,
        step = 0.1
    )

with col2:
    heart_disease = st.selectbox(
        "Heart Disease",
        ["No" , "Yes"]
    )
    heart_disease = 1 if heart_disease == "Yes" else 0

    bmi = st.number_input(
        "BMI",
        min_value=10.0,
        max_value=60.0,
        value=25.0,
        step=0.1
    )

st.subheader("🏢 Lifestyle & Social Factors")
```

```
col1 , col2 = st.columns(2)
with col1:
    work_type_display = st.selectbox(
        "Work Type",
            [
                "Private",
                "Self-employed",
                "Govt job",
                "Children",
                "Never worked"
            ]
        )

    work_type_map = {
        "Private": "Private",
        "Self-employed": "Self-employed",
        "Govt job": "Govt_job",
        "Children": "children",
        "Never worked": "Never_worked"
    }

work_type = work_type_map[work_type_display]
```

```
with col2:
    smoking_status = st.selectbox(
        "Smoking Status",
        [

            "never smoked",
            "formerly smoked",
            "smokes",
            "Unknown"
        ]
    )
```

Figure 7.13: MedInsight Streamlit Patient Screening Interface

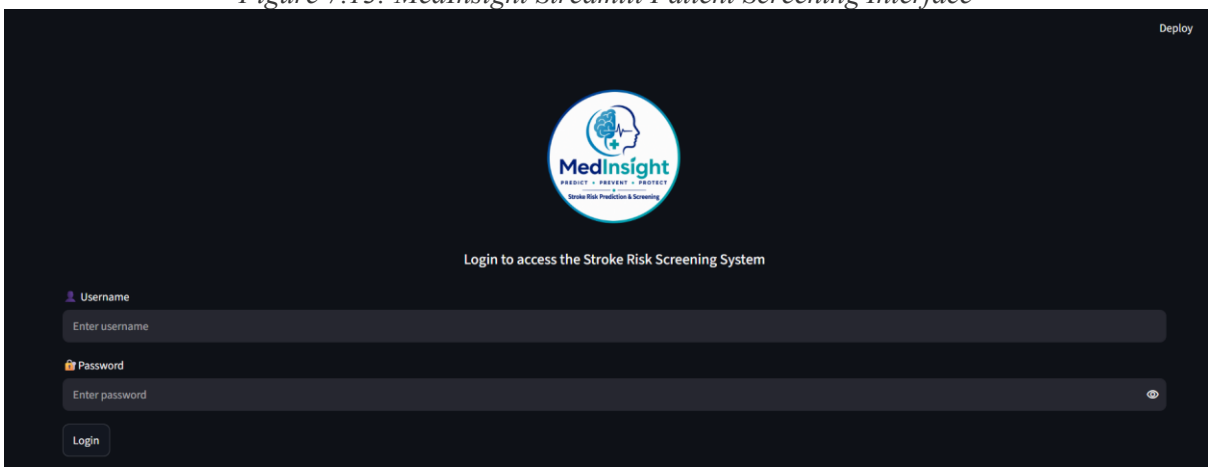


Figure 7.14: MedInsight Login Interface

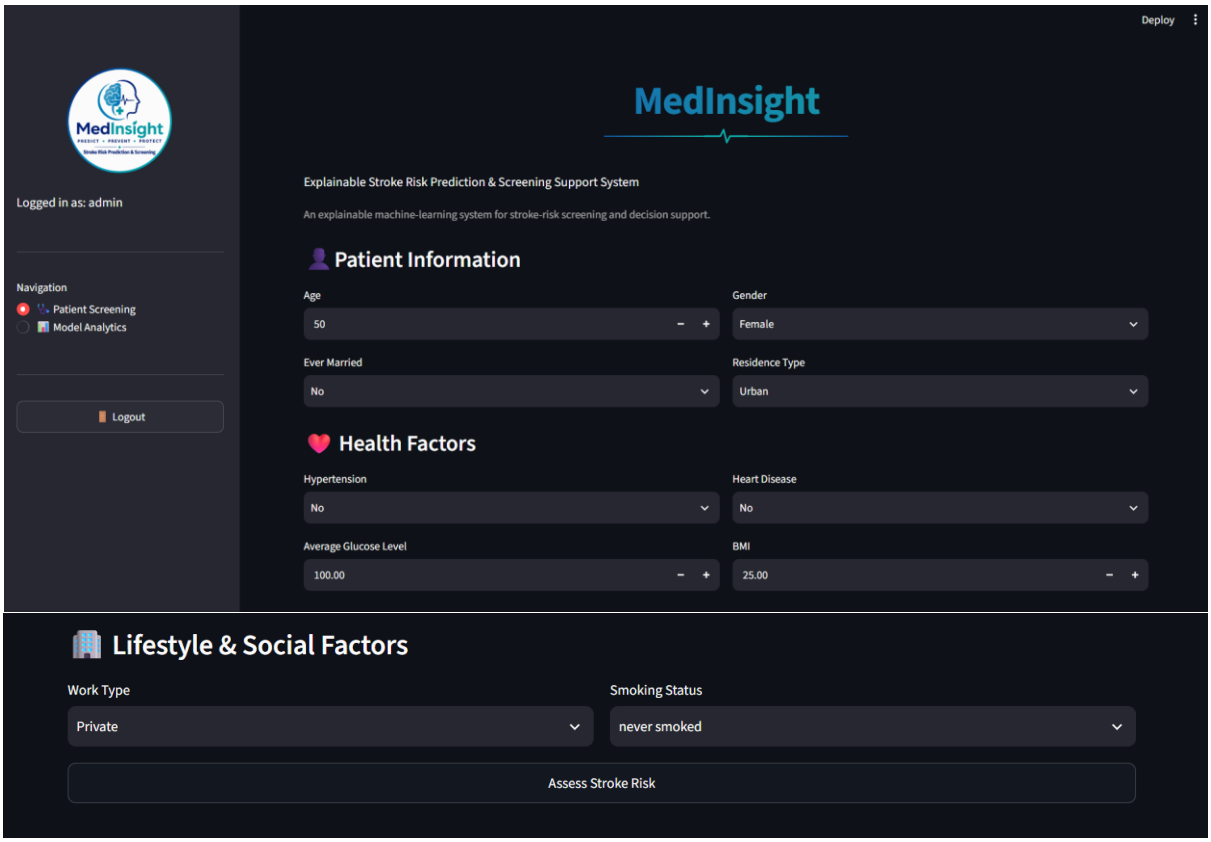


Figure 7.15: MedInsight Patient Screening Interface

Explanation: The MedInsight application is developed using Streamlit and provides a structured interface for stroke-risk screening. The login page provides controlled access to the system. After login, the Patient Screening interface allows users to enter demographic information and health-related factors such as age, gender, hypertension, heart disease, average glucose level and BMI. The sidebar provides navigation between **Patient Screening** and **Model Analytics**.

Module 14 - User Input Processing and Stroke Risk Prediction

Purpose: To transform the entered patient information into the required feature format and generate the stroke-risk prediction using the trained Balanced Logistic Regression model.

```
if st.button("Assess Stroke Risk", use_container_width=True):
    # Convert Work Type into one-hot encoded columns
    work_type_Govt_job = 1 if work_type == "Govt_job" else 0
    work_type_Never_worked = 1 if work_type == "Never_worked" else 0
    work_type_Private = 1 if work_type == "Private" else 0
    work_type_Self_employed = 1 if work_type == "Self-employed" else 0
    work_type_children = 1 if work_type == "children" else 0

    # Convert Smoking Status into one-hot encoded columns
    smoking_status_Unknown = 1 if smoking_status == "Unknown" else 0
    smoking_status_formerly_smoked = 1 if smoking_status == "formerly smoked" else 0
    smoking_status_never_smoked = 1 if smoking_status == "never smoked" else 0
    smoking_status_smokes = 1 if smoking_status == "smokes" else 0
```

```
patient_data = pd.DataFrame({
    'gender': [gender],
    'age': [age],
    'hypertension': [hypertension],
    'heart_disease': [heart_disease],
    'ever_married': [ever_married],
    'Residence_type': [Residence_type],
    'avg_glucose_level': [avg_glucose_level],
    'bmi': [bmi],

    'work_type_Govt_job': [work_type_Govt_job],
    'work_type_Never_worked': [work_type_Never_worked],
    'work_type_Private': [work_type_Private],
    'work_type_Self-employed': [work_type_Self_employed],
    'work_type_children': [work_type_children],

    'smoking_status_Unknown': [smoking_status_Unknown],
    'smoking_status_formerly smoked': [smoking_status_formerly_smoked],
    'smoking_status_never smoked': [smoking_status_never_smoked],
    'smoking_status_smokes': [smoking_status_smokes]
})

# Ensure patient feature order exactly matches the trained model
patient_data = patient_data.reindex(
    columns=X_train.columns,
    fill_value=0
)
```

```
#Display label dict

display_names = {
    "gender": "Gender",
    "age": "Age",
    "hypertension": "Hypertension",
    "heart_disease": "Heart Disease",
    "ever_married": "Ever Married",
    "Residence_type": "Residence Type",
    "avg_glucose_level": "Average Glucose Level",
    "bmi": "BMI",

    "work_type_Govt_job": "Work Type: Govt job",
    "work_type_Never_worked": "Work Type: Never worked",
    "work_type_Private": "Work Type: Private",
    "work_type_Self-employed": "Work Type: Self-employed",
    "work_type_children": "Work Type: Children",

    "smoking_status_Unknown": "Smoking Status: Unknown",
    "smoking_status_formerly smoked": "Smoking Status: Formerly smoked",
    "smoking_status_never smoked": "Smoking Status: Never smoked",
    "smoking_status_smokes": "Smoking Status: Smokes"
}
```

```
#Make Prediction
prediction = best_balanced_lr.predict(patient_data)[0] #stroke / no stroke

#Get Probability
probability = best_balanced_lr.predict_proba(patient_data)[0][1] #estimate stroke probability
```

Figure 7.16: Stroke Risk Assessment Result

Explanation: After the user clicks **"Assess Stroke Risk"**, the selected categorical values are converted into numerical and one-hot encoded features. These values are assembled into a Pandas DataFrame and reordered to match the feature structure used during model training. The prepared patient data is then passed to the saved Balanced Logistic Regression model to obtain the predicted class and estimated stroke probability.

Module 15 - Result Display and Risk Assessment

Purpose: To display the model's prediction and estimated stroke probability in a clear and understandable form for the user.

```
st.subheader("🩺Risk Assessment")
if probability >= 0.50:
    st.error("🔴 Higher Stroke Risk Detected")
elif probability >= 0.20:
    st.warning("🟡 Moderate Stroke Risk Detected")
else:
    st.success("🟢 Lower Stroke Risk Detected")
st.metric(
    "Estimated Stroke Probability",
    f"{probability * 100:.2f}%"
)
# st.write(f"Estimated Stroke Probability: {probability * 100:.2f}%")
st.info(
"This tool is intended for screening and decision-support purposes only "
"and is not a substitute for professional medical advice or diagnosis."
)
```

Figure 7.17: Stroke Risk Assessment and Prediction Result

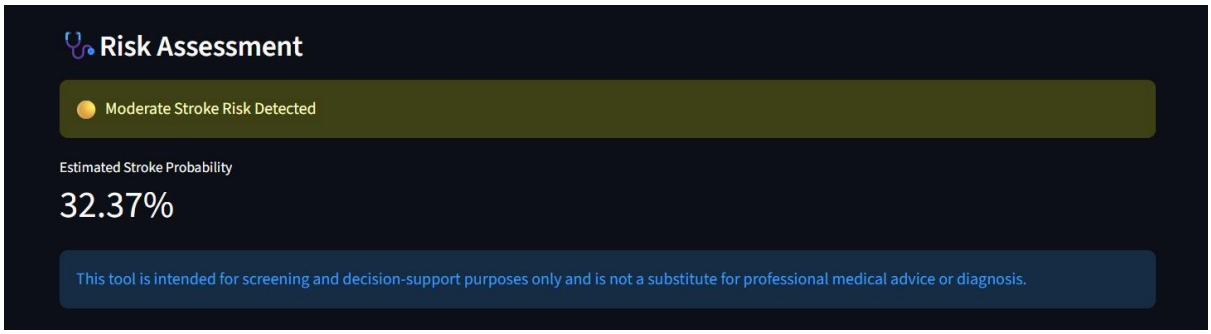


Figure 7.17: MedInsight Stroke Risk Assessment Output

Explanation: After generating the prediction and probability, the application classifies the estimated risk into three levels: **Higher, Moderate or Lower Stroke Risk**. The corresponding message is displayed using Streamlit's error, warning, or success components. The estimated stroke probability is also displayed as a percentage. A screening disclaimer is

provided to clarify that the result is intended for screening and decision-support purposes and is not a medical diagnosis.

Module 16 - SHAP Explainability & Prediction Rationale

Purpose: To explain how individual patient features contribute to the model's stroke-risk prediction.

```
#Create SHAP explainer
#calculates how much each feature contributed to this patient's prediction.
sample_shap = explainer(patient_data)
#Display
st.subheader("Prediction Rationale")
st.caption(
    "The chart below shows how each patient feature influenced "
    "the model's prediction."
)
st.caption(
    "🟥 Red = increases the model output  |  "
    "🟦 Blue = decreases the model output"
    )

display_shap = sample_shap[0]
```

```
# Display SHAP waterfall chart
shap.plots.waterfall(
    display_shap,
    show=False
)

ax = plt.gca()
# Remove automatic encoded value text from labels
labels = []

for label in ax.get_yticklabels():
    text = label.get_text()

    if "=" in text:
        text = text.split("=", 1)[1].strip()

    labels.append(text)
ax.set_yticklabels(labels)
st.pyplot(plt.gcf())
```

```
# Get the indices of the 5 most influential features
top_indices = sorted(
    range(len(shap_values)),
    key=lambda i: abs(shap_values[i]),
    reverse=True
)[:5]
```

```
contribution_df = pd.DataFrame(contribution_data)
st.dataframe(
    contribution_df,
    use_container_width=True,
    hide_index=True
)
```

Note: The code snippets presented above show the key implementation steps of SHAP explainability. The remaining code is used for feature-label formatting, filtering relevant SHAP features, preparing user-friendly contribution details, and formatting the displayed output in the Streamlit interface.

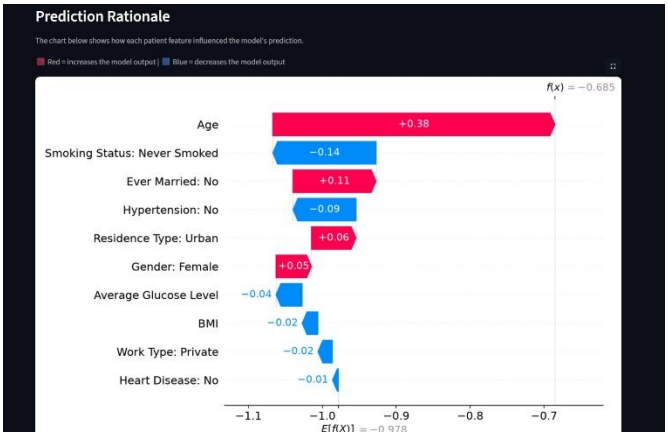


Figure 7.18: SHAP Prediction Rationale

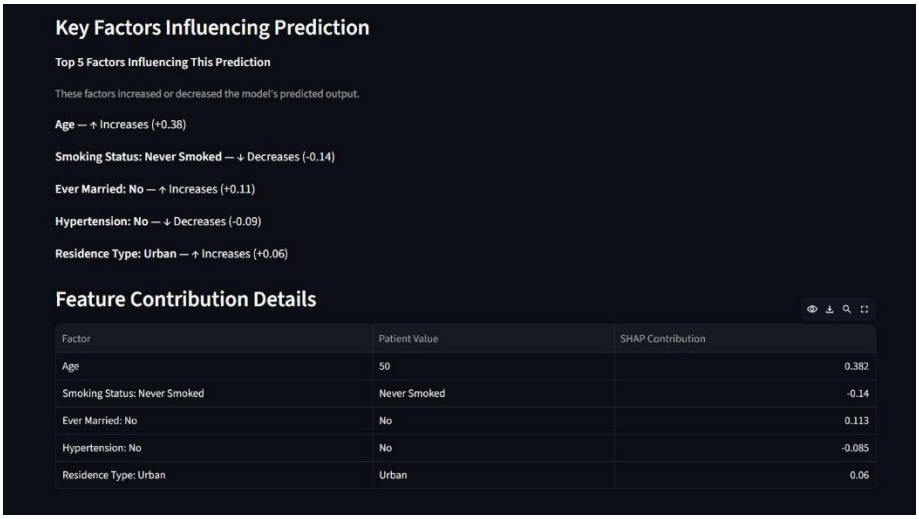


Figure 7.19: Key Factors Influencing the Prediction

Explanation: SHAP is used to calculate the contribution of each feature to the model's prediction. The application generates a **SHAP waterfall plot** to show which features increase or decrease the model output. It also identifies the **top five influential factors** and displays their SHAP contribution values, making the prediction easier to interpret.

Module 17 - Risk-Factor Recommendations and Decision Support

Purpose: To provide personalised risk-factor guidance based on the patient's predicted stroke risk and entered health information.

```
# PERSONALIZED STROKE RISK GUIDANCE
#=========================================

    st.markdown("---")
    st.subheader("🩺 What Should You Focus On?")

    guidance = []
```

```python
if probability >= 0.50:
    st.error(
        "⚠️ **Higher estimated stroke risk:** "
        "The model indicates a higher estimated risk based on the information provided. "
        "Focus on controlling modifiable risk factors and arrange appropriate medical follow-up."
    )

elif probability >= 0.20:
    st.warning(
        "⚠️ **Moderate estimated stroke risk:** "
        "The model indicates an intermediate level of estimated risk. "
        "Pay particular attention to your modifiable risk factors and maintain regular health checkups."
    )

else:
    st.success(
        "💚 **Lower estimated stroke risk:** "
        "The model indicates a lower estimated risk based on the information provided. "
        "Continue maintaining healthy habits and regular preventive checkups."
    )
```

```python
# ------------------------------------------------------
# 5. BMI / WEIGHT
# ------------------------------------------------------
# Dynamic BMI-based guidance
if bmi >= 30:
    weight_guidance = (
        "Your BMI is in the obesity range. "
        "Focus on gradual, sustainable weight management through "
        "balanced nutrition and regular physical activity appropriate for your health."
    )
elif bmi >= 25:
    weight_guidance = (
        "Your BMI is in the overweight range. "
        "Focus on balanced nutrition, regular physical activity, "
        "and gradual, sustainable weight management."
    )
else:
    weight_guidance = (
        "Your BMI is within the healthy range. "
        "Continue maintaining balanced nutrition and regular physical activity."
    )

guidance.append(f"⚖️ **Weight Management:** {weight_guidance}")
```

```python
# ------------------------------------------------------------
# DISPLAY GUIDANCE
# ------------------------------------------------------------

st.markdown("### 🎯 Recommended Focus Areas")

for item in guidance:
    st.markdown(f"- {item}")
```

Figure 7.20: Personalised Risk-Factor Recommendations and Decision Support

Note: The code snippets above represent the core logic used for risk-based messaging and dynamic recommendation generation. Similar conditional rules are implemented for hypertension, heart disease, glucose level, smoking status, physical activity and regular follow-up. The complete implementation contains these additional rules and their corresponding guidance messages.

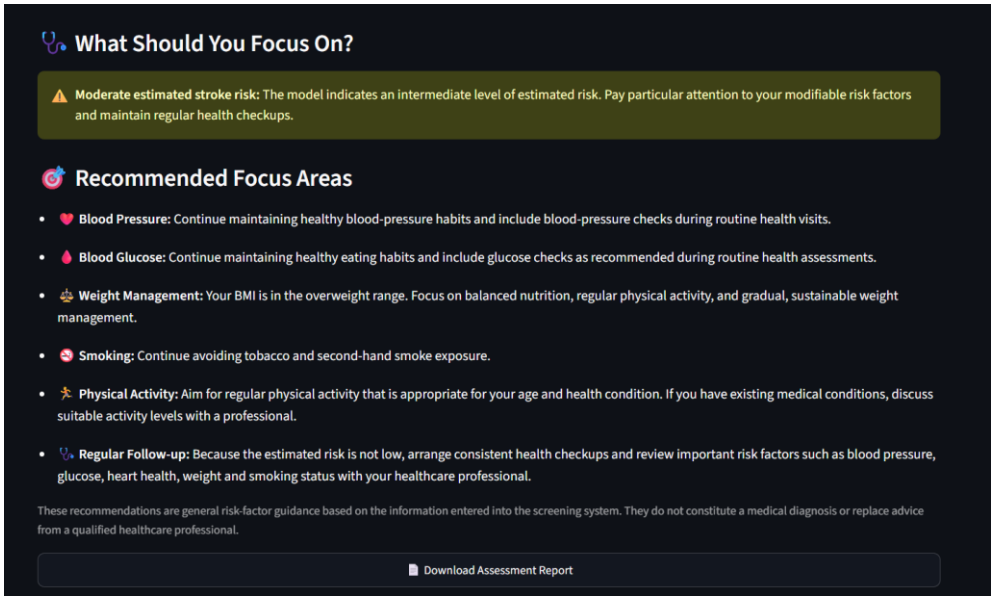


Figure 7.21: Risk-Factor Recommendations and Decision Support

Explanation: After the risk assessment, the system generates an overall risk message based on the estimated stroke probability. It then provides personalised focus areas related to **blood pressure, blood glucose, weight management, smoking, physical activity and regular follow up**. The recommendations are generated using conditional rules based on the patient's entered values. This helps the user understand which modifiable risk factors require attention.
The module also displays a disclaimer stating that the recommendations are **general risk factor guidance** and do not constitute a medical diagnosis or replace advice from a qualified healthcare professional.

Module 18 - PDF Assessment Report Generation

Purpose: To generate a downloadable PDF report containing the patient's stroke-risk assessment and relevant prediction details.

```
def create_pdf_report(
    age,
    gender,
    ever_married,
    Residence_type,
    hypertension,
    heart_disease,
    avg_glucose_level,
    bmi,
    work_type,
    smoking_status,
    probability,
    prediction,
    guidance
):
    buffer = BytesIO()

    doc = SimpleDocTemplate(
        buffer,
        pagesize=A4,
        rightMargin=40,
        leftMargin=40,
        topMargin=40,
        bottomMargin=40
    )
```

Note: This defines the PDF-report function and creates an in-memory PDF document using BytesIO and SimpleDocTemplate**.**

```
story = []
# Header
story.append(
    Paragraph("MEDINSIGHT", title_style)
)
story.append(
    Paragraph(
        "Stroke Risk Screening & Explainable ML Report",
        subtitle_style
    )
)
story.append(ECGLine())
story.append(Spacer(1, 12))


patient_table = Table([
    ["Age", str(age)],
    ["Gender", gender],
    ["Ever Married", ever_married],
    ["Residence Type", Residence_type]
], colWidths=[105, 120])
```

```
if probability >= 0.50:

    risk_result = "Higher Predicted Risk"
    risk_color = "#C62828"

    risk_explanation = (
        "Based on the patient's information, "
        "the machine learning model predicts a higher "
        "likelihood of stroke."
    )

elif probability >= 0.20:

    risk_result = "Moderate Predicted Risk"
    risk_color = "#B8860B"

    risk_explanation = (
        "Based on the patient's information, "
        "the machine learning model predicts a moderate "
        "likelihood of stroke."
    )

else:

    risk_result = "Lower Predicted Risk"
    risk_color = "#247A45"

    risk_explanation = (
        "Based on the patient's information, "
        "the machine learning model predicts a lower "
        "likelihood of stroke."
    )
```

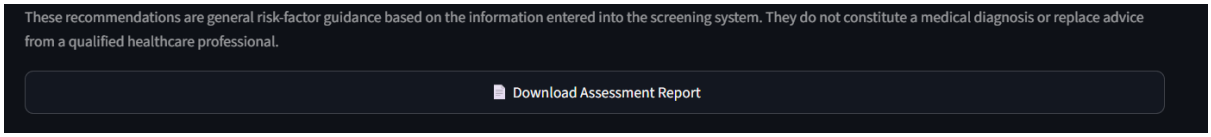
```
    )
    doc.build(story)
    buffer.seek(0)
    return buffer
```

```
pdf_report = create_pdf_report(
age=age,
gender="Female" if gender == 0 else "Male" if gender == 1 else "Other",
ever_married="Yes" if ever_married == 1 else "No",
Residence_type="Urban" if Residence_type == 1 else "Rural",
hypertension="Yes" if hypertension == 1 else "No",
heart_disease="Yes" if heart_disease == 1 else "No",
avg_glucose_level=avg_glucose_level,
bmi=bmi,
work_type=work_type,
smoking_status=smoking_status,
probability=probability,
prediction=prediction,
guidance = guidance
)
```

```
st.download_button(
label="📄 Download Assessment Report",
data= pdf_report,
file_name="MedInsight_Assessment_Report.pdf",
mime="application/pdf",
use_container_width=True
)
```

Figure 7.22: Downloadable Stroke Risk Assessment Report

Note: The completed report is generated in memory and returned as a PDF buffer. Streamlit's st.download_button() allows the user to download the generated assessment report.



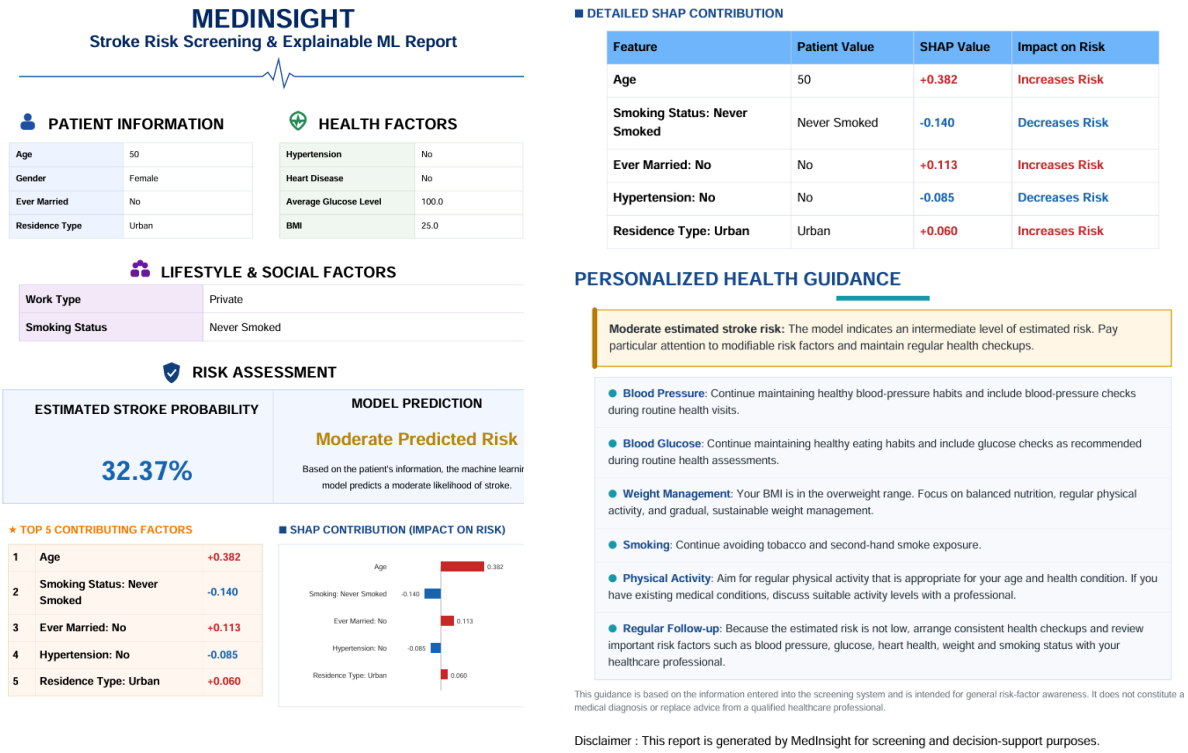


Figure 7.23: Generated MedInsight Stroke Risk Assessment Report

Explanation: The system uses **ReportLab** to generate a structured assessment report. The PDF can include the patient's assessment information, predicted stroke risk, estimated probability, key contributing factors and risk factor recommendations. The generated report can then be downloaded directly from the Streamlit application using the **Download Assessment Report** button.

Module 19 - Model Analytics Dashboard

Purpose: To provide a separate interface for viewing and comparing the performance of the trained machine learning models.

```
if page == "📊 Model Analytics":
    st.title("📊 Model Analytics")
    # st.write("Model performance and analytics will be shown here.")
    st.subheader("Final Model")
    st.info(
        "Tuned Balanced Logistic Regression\n\n"
        "Selected as the final model after model comparison, "
        "class balancing and hyperparameter tuning."
    )
    #---------------------------------
    st.subheader("Model Performance")
    #Predictions from final tuned balanced LR model
    y_pred_analytics = best_balanced_lr.predict(X_test)

    # Calculate performance metrics
    accuracy = accuracy_score(y_test, y_pred_analytics)
    precision = precision_score(y_test, y_pred_analytics, zero_division=0)
    recall = recall_score(y_test, y_pred_analytics, zero_division=0)
    f1 = f1_score(y_test, y_pred_analytics, zero_division=0)

    # Display metrics
    col1, col2, col3, col4 = st.columns(4)
    with col1:
        st.metric("Accuracy", f"{accuracy * 100:.2f}%")
    with col2:
        st.metric("Precision", f"{precision * 100:.2f}%")
    with col3:
        st.metric("Recall", f"{recall * 100:.2f}%")
    with col4:
        st.metric("F1 Score", f"{f1 * 100:.2f}%")

    st.subheader("Model Comparison")
    comparison_display = pd.DataFrame({
        "Model": [
            "Logistic Regression",
            "KNN",
            "Decision Tree",
            "Random Forest",
            "SMOTE + Logistic Regression",
            "Balanced Logistic Regression"
        ],
        "Accuracy": [
            95.21, 94.81, 91.10, 94.91, 93.25, 74.66
        ],
        "Recall": [
            2.00, 2.00, 20.00, 2.00, 38.00, 80.00
        ],
        "Precision": [
            100.00, 20.00, 16.39, 25.00, 33.33, 13.84
        ],
        "F1 Score": [
            3.92, 3.64, 18.02, 3.70, 35.51, 23.60
        ]
    }),
    st.dataframe(
        comparison_display,
        hide_index=True,
        use_container_width=True
    )
```

Figure 7.24: Model Analytics Dashboard

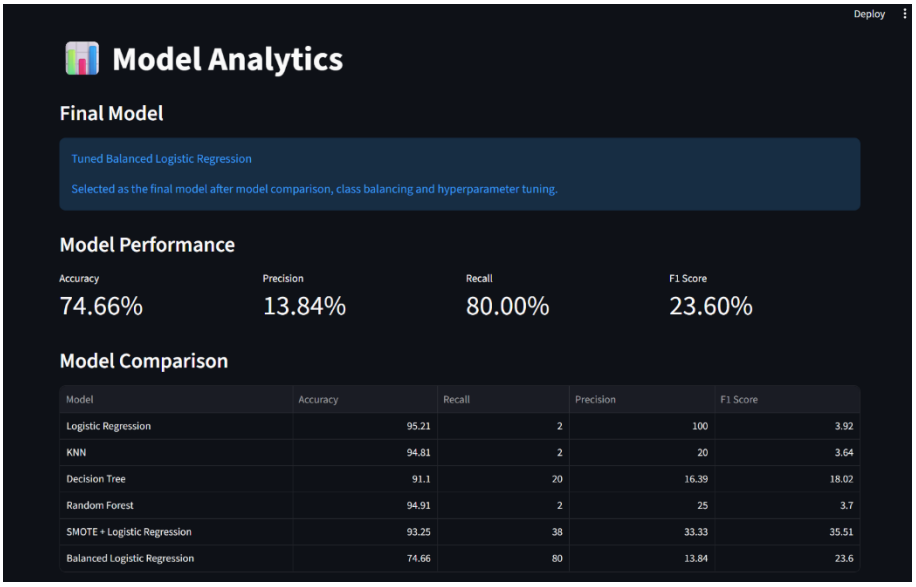


Figure 7.25: MedInsight Model Analytics Dashboard

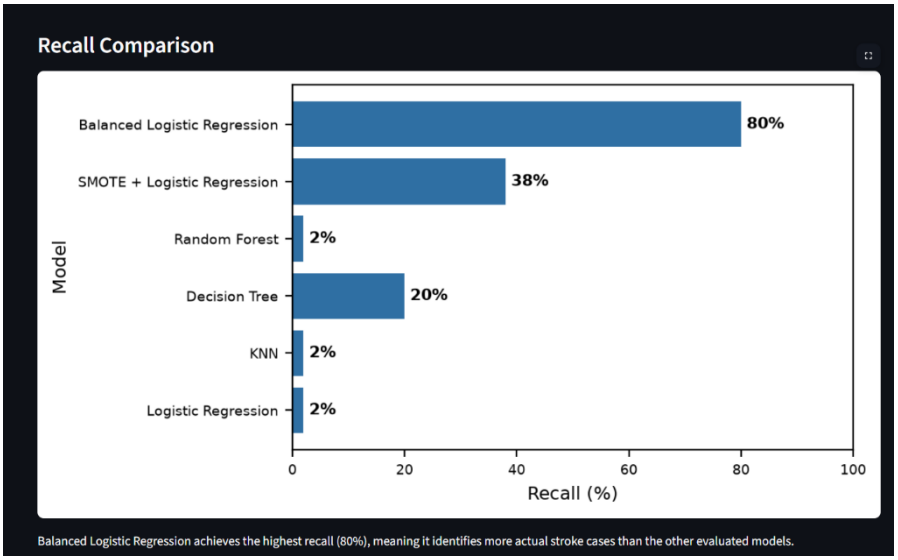


Figure 7.26: Recall Comparison of Evaluated Models

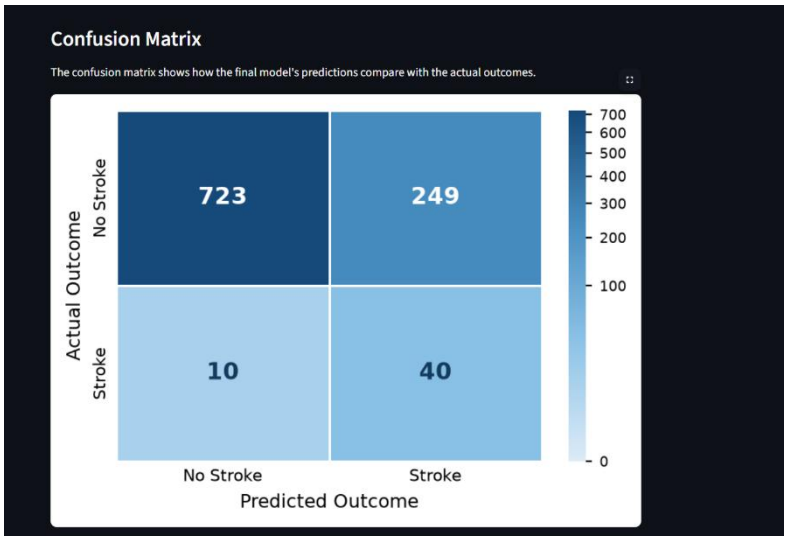


Figure 7.27: Confusion Matrix of the Final Model

Explanation: The Model Analytics Dashboard displays the **final selected model** along with its key performance metrics, including **Accuracy, Precision, Recall and F1-score**. It also provides a model comparison table containing the performance of Logistic Regression, KNN, Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression.

The dashboard further presents a **confusion matrix** to show the relationship between actual and predicted stroke outcomes. A **recall comparison chart** is also provided to compare how effectively the evaluated models identify actual stroke cases. These visualizations support the evaluation and interpretation of the final model.

CHAPTER 8 –DATASET AND DATA VISUALIZATION

8.1 Dataset Description

- **Name:** Stroke Prediction Dataset
- **Source:** Kaggle
- **Total Records:** 5110 patient records
- **Total Features:** 10 input features and 1 target variable (stroke) after removing the id column
- **Target Variable**: stroke (0 = No Stroke, 1 = Stroke)
- **Missing Values**: Missing values are present in the bmi attribute and are handled by replacing them with the median BMI value.

8.2 Dataset Summary

Dataset	Summary
Total Samples	5110
No Stroke Cases	4861
Stroke Cases	249
Input Features	10 patient related features
Feature Types	Numerical and categorical features
Target Variable	stroke (0 = No Stroke, 1 = Stroke)
Class Distribution	Imbalanced, with substantially fewer stroke cases than no-stroke cases

Interpretation: The dataset contains 5,110 patient records with demographic, medical and lifestyle-related attributes. Among these, 4,861 records correspond to no stroke cases and 249 correspond to stroke cases, indicating a significant class imbalance. The dataset contains 10 input features consisting of numerical and categorical attributes, while stroke is used as the binary target variable. The observed class imbalance is considered during model development through appropriate class-balancing techniques.

8.3 Correlation Heatmap

A correlation heatmap was generated using the **Seaborn library** to visualize the relationships among the numerical features in the stroke prediction dataset. It helps identify the strength and direction of relationships between the numerical variables, providing useful insights into the dataset before training the machine learning models.

```
correlation_data = df.select_dtypes(include=['number']).drop(columns = ['id'])

plt.figure(figsize=(10, 7))
sns.heatmap(
    correlation_data.corr(),
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)
plt.title(
    'Correlation Heatmap of Numerical Features',
    fontsize=14,
    fontweight='bold'
)

plt.tight_layout()
plt.savefig(
    'correlation_heatmap.png',
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```

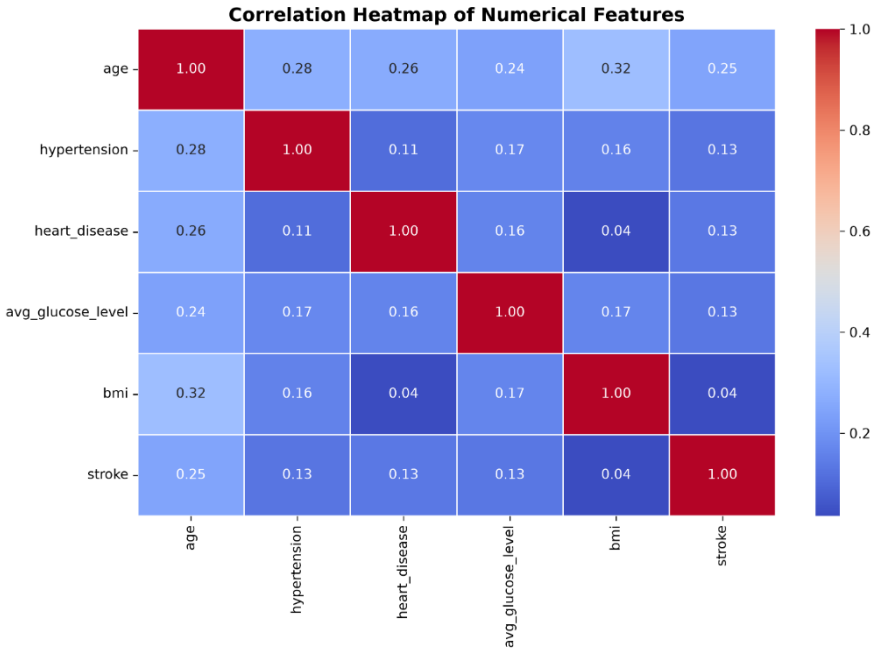


Figure 8.1: Correlation Heatmap of Numerical Features

8.4 Stroke Class Distribution

A bar chart was generated to visualize the distribution of stroke and non-stroke cases in the dataset. This visualization helps identify the class distribution and highlights the imbalance between the two classes.

```
stroke_counts = df['stroke'].value_counts().sort_index()
plt.figure(figsize=(7, 5))
bars = plt.bar(
    ['No Stroke', 'Stroke'],
    stroke_counts.values
)
plt.title(
    'Stroke Class Distribution',
    fontsize=14,
    fontweight='bold'
)
plt.xlabel('Stroke Status')
plt.ylabel('Number of Patients')

# Add values above bars
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f'{int(height)}',
        ha='center',
        va='bottom',
        fontsize=10
    )
plt.tight_layout()
plt.savefig(
    'stroke_class_distribution.png', dpi=300, bbox_inches='tight')
plt.show()
```

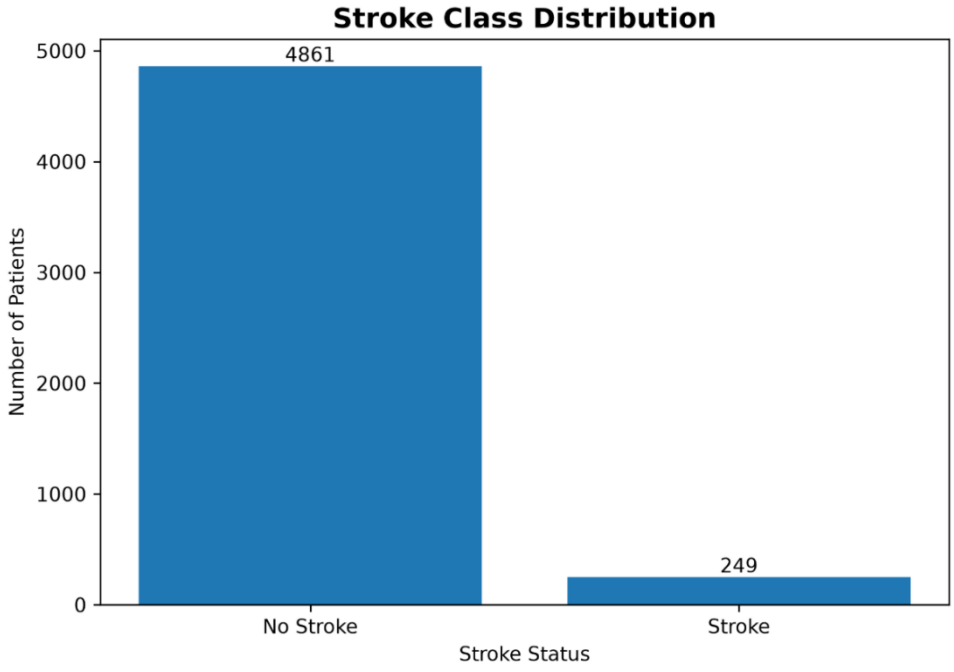


Figure 8.2: Stroke Class Distribution

Interpretation: The chart shows that the dataset contains a substantially larger number of **non-stroke cases (4,861)** compared with **stroke cases (249)**. This indicates a significant class imbalance which was considered during model development through class-balancing techniques such as SMOTE and balanced Logistic Regression.

8.5 Feature - wise Stroke Distribution

Several patient related features were analyzed to observe their distribution across stroke and non-stroke cases. Bar charts were generated for important medical risk factors, including hypertension, heart disease and average glucose level. These visualizations provide a clearer understanding of how stroke cases are distributed across different risk factor categories.

8.5.1 Hypertension vs Stroke

```
hypertension_stroke = pd.crosstab(
    df['hypertension'],
    df['stroke']
)
hypertension_stroke.columns = ['No Stroke', 'Stroke']
hypertension_stroke.index = ['No Hypertension', 'Hypertension']
hypertension_stroke.plot(
    kind='bar',
    figsize=(8, 5)
)
plt.title(
    'Stroke Distribution by Hypertension',
    fontsize=14,
    fontweight='bold'
)

plt.xlabel('Hypertension Status')
plt.ylabel('Number of Patients')
plt.xticks(rotation=0)
plt.legend(title='Stroke Status')
plt.tight_layout()
plt.savefig(
    'hypertension_vs_stroke.png',
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```

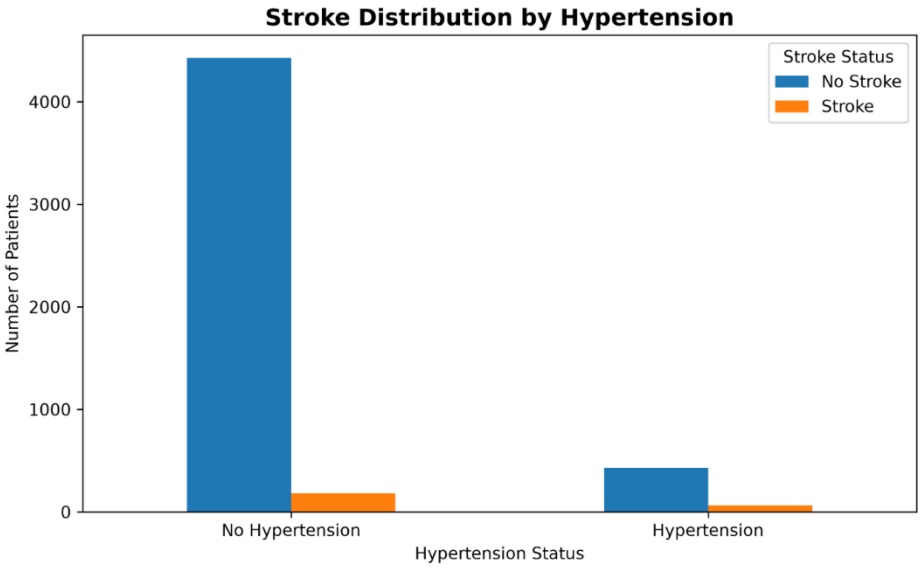


Figure 8.3: Stroke Distribution by Hypertension

Interpretation: The visualization compares stroke and non-stroke cases according to hypertension status. It shows that stroke cases are present in both groups, while the number of non-stroke cases is considerably higher. The comparison helps examine hypertension as an important health-related factor in the dataset.

8.5.2 Heart Disease vs Stroke

```
heart_stroke = pd.crosstab(
    df['heart_disease'],
    df['stroke']
)

heart_stroke.columns = ['No Stroke', 'Stroke']

heart_stroke.index = ['No Heart Disease', 'Heart Disease']

heart_stroke.plot(
    kind='bar',
    figsize=(8, 5)
)
plt.title(
    'Stroke Distribution by Heart Disease',
    fontsize=14,
    fontweight='bold'
)
plt.xlabel('Heart Disease Status')
plt.ylabel('Number of Patients')
plt.xticks(rotation=0)
plt.legend(title='Stroke Status')
plt.tight_layout()
plt.savefig(
    'heart_disease_vs_stroke.png',
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```

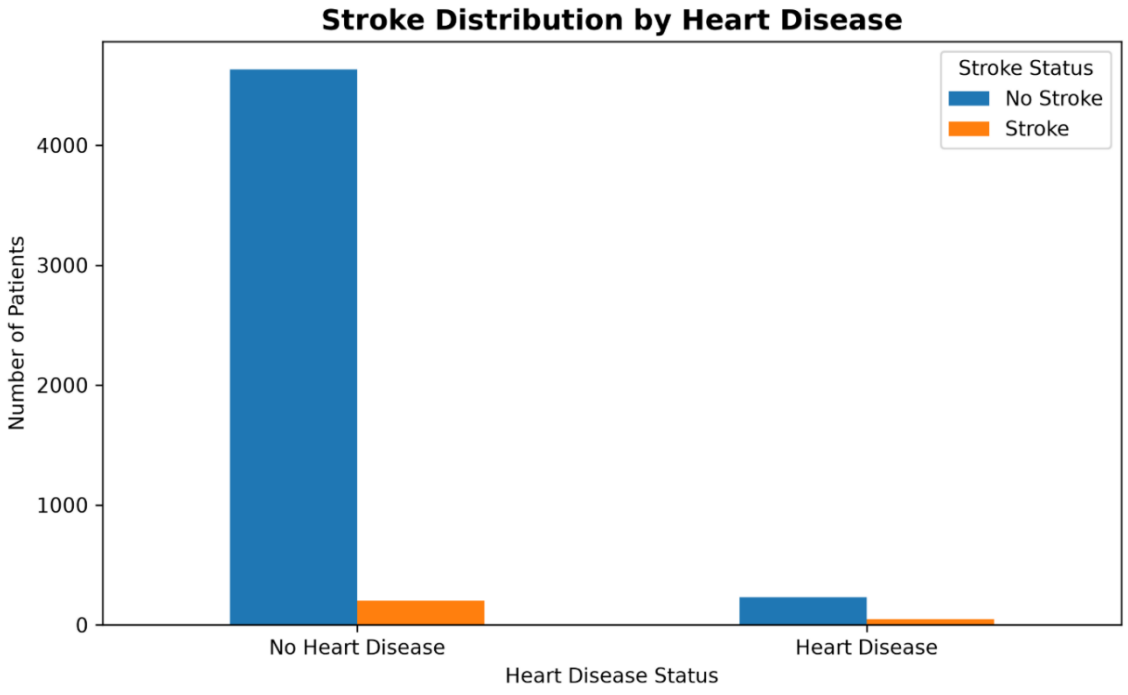


Figure 8.4: Stroke Distribution by Heart Disease

Interpretation: The chart compares stroke and non-stroke cases according to heart disease status. It shows that stroke cases occur in both groups, while non-stroke cases are considerably more frequent. The visualization helps examine the distribution of stroke cases in relation to the presence or absence of heart disease.

8.5.3 Average Glucose Level vs Stroke

```
glucose_df = df.copy()
glucose_df['glucose_category'] = pd.cut(
    glucose_df['avg_glucose_level'],
    bins=[0, 100, 140, float('inf')],
    labels=['Normal', 'Elevated', 'High']
)
glucose_stroke = pd.crosstab(
    glucose_df['glucose_category'],
    glucose_df['stroke']
)
glucose_stroke.columns = ['No Stroke', 'Stroke']
glucose_stroke.plot(
    kind='bar',
    figsize=(8, 5)
)
plt.title(
    'Stroke Distribution by Average Glucose Level',
    fontsize=14,
    fontweight='bold'
)
plt.xlabel('Average Glucose Level')
plt.ylabel('Number of Patients')
plt.xticks(rotation=0)
plt.legend(title='Stroke Status')
plt.tight_layout()
plt.savefig(
    'glucose_vs_stroke.png',
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```

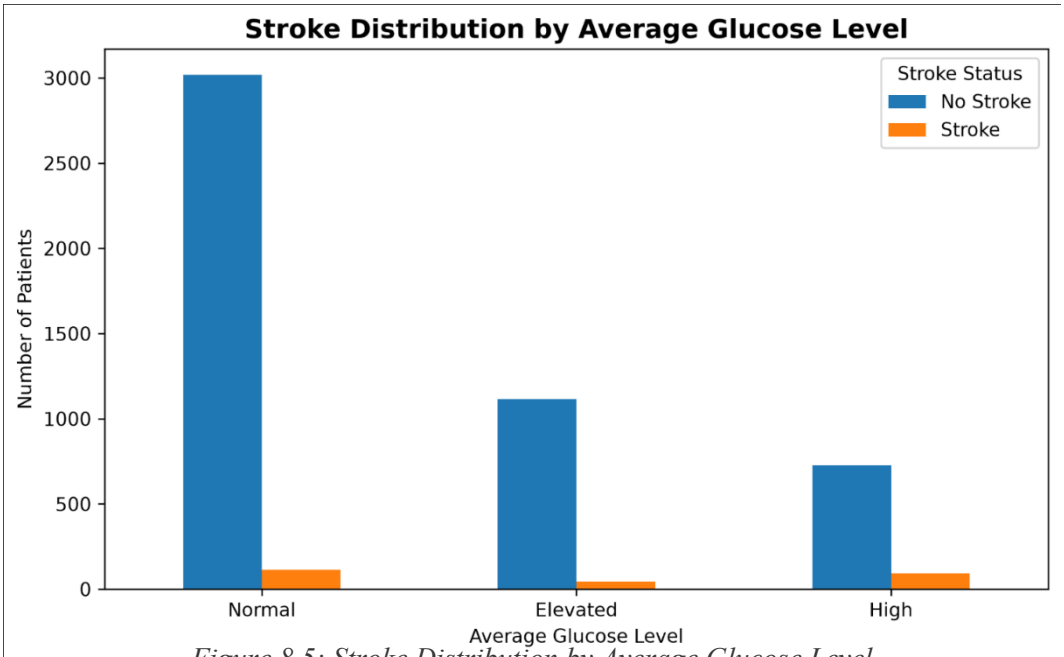


Figure 8.5: Stroke Distribution by Average Glucose Level

Interpretation: The chart compares stroke and non-stroke cases across different average glucose level categories. It shows the distribution of stroke cases among normal, elevated and high glucose level groups. This visualization helps examine the relationship between glucose-level categories and stroke occurrence in the dataset.

Additional visualizations were also generated for gender, BMI category and smoking status but only the most relevant feature-wise distributions are presented here for concise analysis.

8.6 Scatter Plot

A scatter plot was generated to examine the relationship between **age** and **average glucose level** while distinguishing between stroke and non-stroke cases.

```
plt.figure(figsize=(8, 6))
for stroke_value in [0, 1]:
    subset = df[df["stroke"] == stroke_value]
    plt.scatter(
        subset["age"],
        subset["avg_glucose_level"],
        label=f"Stroke = {stroke_value}",
        alpha=0.5
    )
plt.xlabel("Age")
plt.ylabel("Average Glucose Level")
plt.title("Age vs Average Glucose Level by Stroke Status")
plt.legend()
plt.tight_layout()
plt.savefig("age_vs_glucose_scatter.png", dpi=300)
plt.show()
```

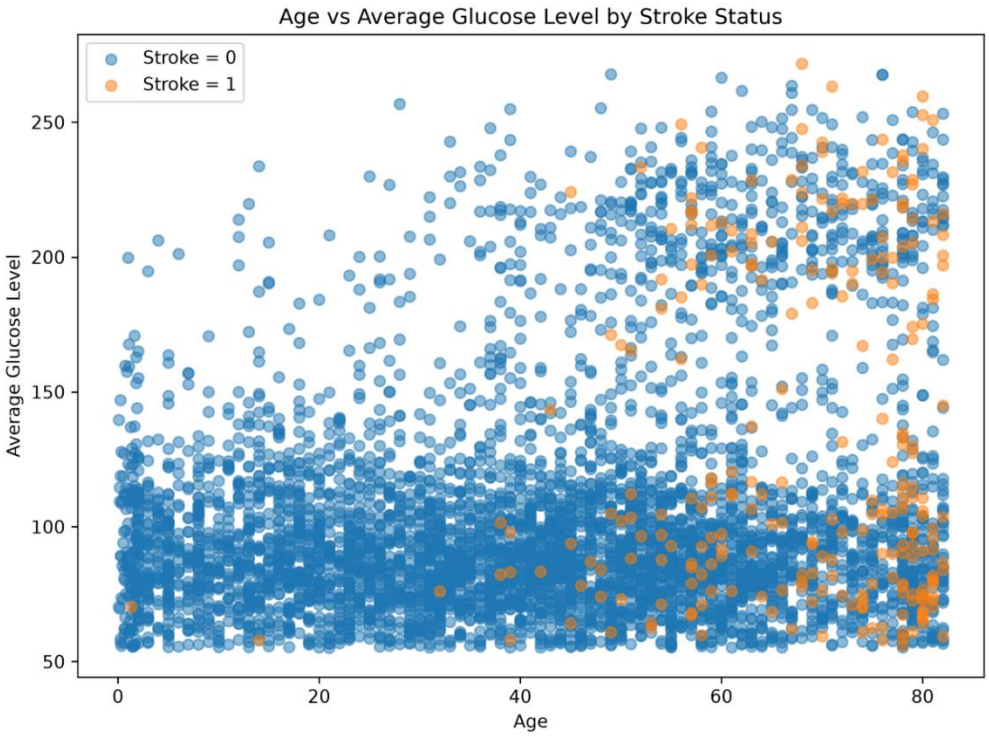


Figure 8.6: Age vs Average Glucose Level by Stroke Status

8.7 Age Distribution of Patients

```
plt.figure(figsize=(8, 5))
plt.hist(
    df['age'],
    bins=20
)
plt.title(
    'Age Distribution of Patients',
    fontsize=14,
    fontweight='bold'
)
plt.xlabel('Age')
plt.ylabel('Number of Patients')
plt.tight_layout()
plt.savefig(
    'age_distribution.png',
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```

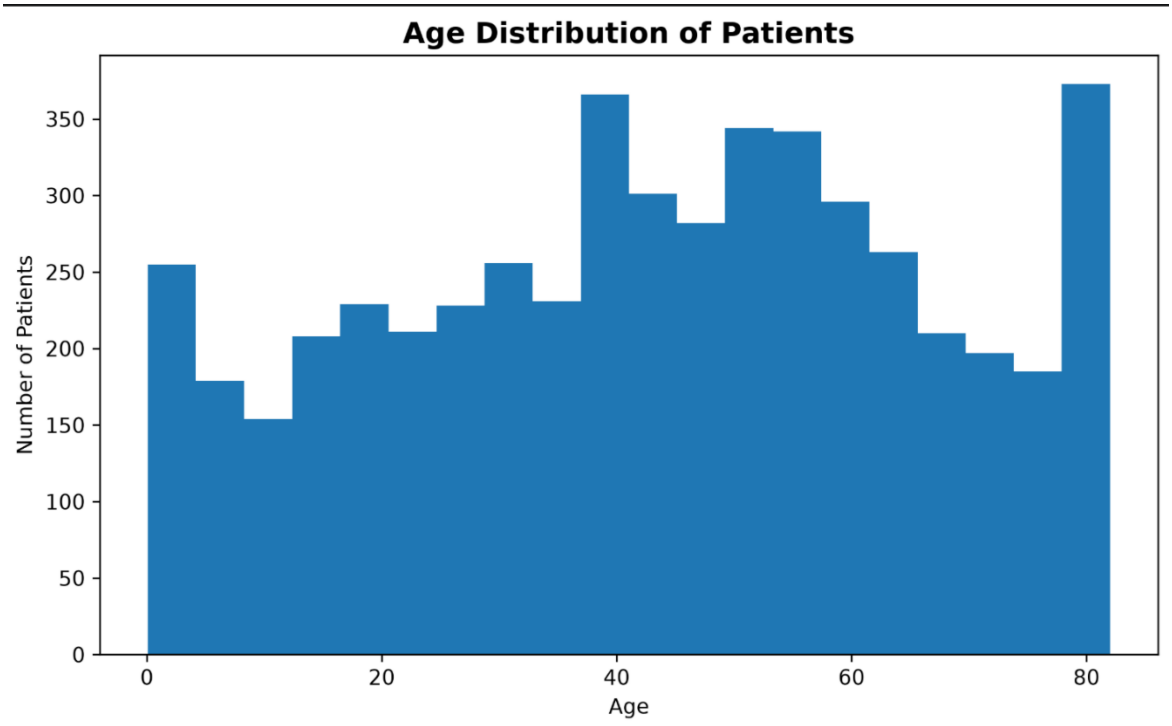


Figure 8.7: Age Distribution of Patients

8.8 Best Model for Prediction: Tuned Balanced Logistic Regression

Tuned Balanced Logistic Regression was selected as the final model for stroke-risk prediction. The evaluated models included Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression. The model was further tuned through hyperparameter optimization to improve its performance and obtain a more suitable configuration for the imbalanced

stroke dataset. Although some models achieved higher accuracy, the Tuned Balanced Logistic Regression achieved the highest recall of 80%, making it more suitable for identifying stroke cases. Therefore it was selected as the final prediction model for the MedInsight screening system.

```python
# Hyperparameter tuning for Balanced Logistic Regression

lr_balanced_tune = LogisticRegression(
    class_weight='balanced',
    max_iter=2000
)
param_grid_balanced = {
    'C': [0.01, 0.1, 1, 10, 100]
}
grid_balanced = GridSearchCV(
    lr_balanced_tune,
    param_grid_balanced,
    cv=cv,
    scoring='f1',
    n_jobs=-1
)
grid_balanced.fit(X_train, y_train)

print("Best C:", grid_balanced.best_params_)
print("Best CV F1:", grid_balanced.best_score_)

# Best tuned Balanced Logistic Regression
best_balanced_lr = grid_balanced.best_estimator_
```

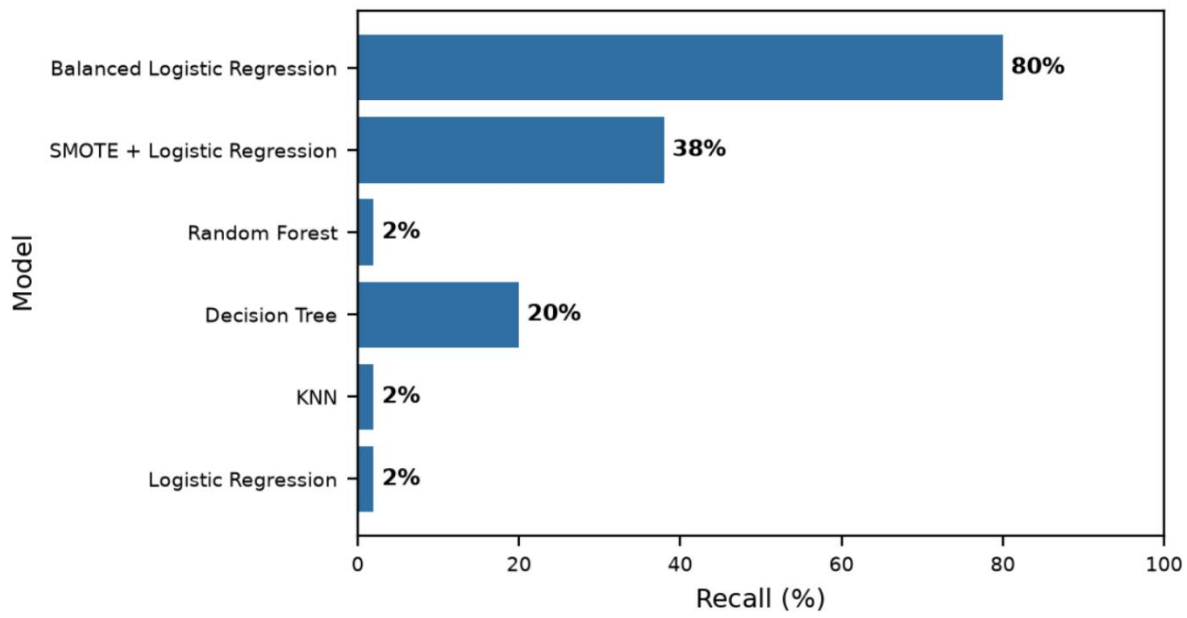


Figure 8.8: Recall Comparison of Evaluated Models

Interpretation: The recall comparison shows that Balanced Logistic Regression achieves the highest recall of **80%** among the evaluated models. This indicates that it identifies a larger proportion of the actual stroke cases, supporting its selection as the final model for the MedInsight screening system.

CHAPTER 9 - RESULTS AND DISCUSSION

9.1 Model Performance Comparison

Several machine learning models were evaluated for stroke-risk prediction using the MedInsight dataset. The evaluated models included Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression. Their performance was compared using Accuracy, Precision, Recall and F1-score. The comparison helps identify how effectively each model can classify stroke and non-stroke cases, particularly in the presence of class imbalance.

Table 9.1: Performance Comparison of Evaluated Models

Model	Accuracy(%)	Precision (%)	Recall (%)	F1-score(%)
Logistic Regression	95.21	100.00	2.00	3.92
KNN	94.81	20.00	2.00	3.64
Decision Tree	91.10	16.39	20.00	18.02
Random Forest	94.91	25.00	2.00	3.70
SMOTE + Logistic Regression	93.25	33.33	38.00	35.51
Balanced Logistic Regression	74.66	13.84	80.00	23.60

The comparison shows that the conventional models achieved high overall accuracy but generally produced very low recall values for stroke cases. In contrast **Balanced Logistic Regression achieved a substantially higher recall of 80.00%** although its accuracy was lower at 74.66%. This indicates that **class balancing improved the model's ability to identify positive stroke cases, which is particularly important for a stroke-risk screening application.**

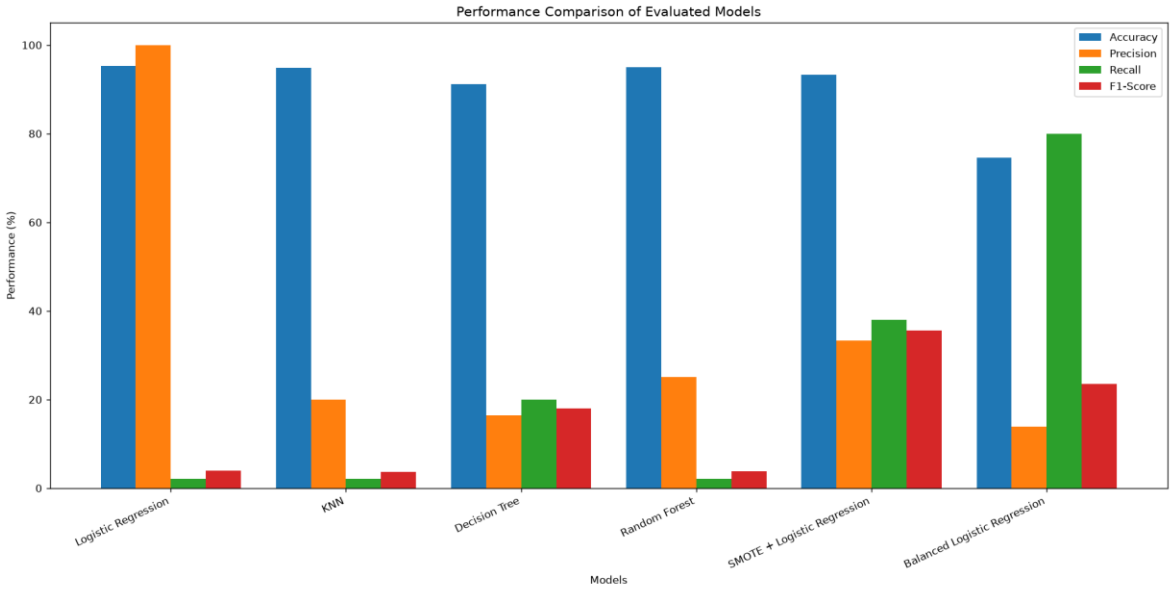


Figure 9.1: Performance Comparison of Evaluated Models

The comparison shows that the conventional models achieved high accuracy but generally produced very low recall for stroke cases. In contrast, the Balanced Logistic Regression model achieved the highest recall of **80.00%**, although its accuracy was lower at **74.66%**. The SMOTE-based Logistic Regression model also improved recall to **38.00%** compared with the **2.00%** recall of conventional Logistic Regression. These results indicate that accuracy alone is insufficient for evaluating stroke-risk prediction, and greater importance should be given to the model's ability to identify actual stroke cases.

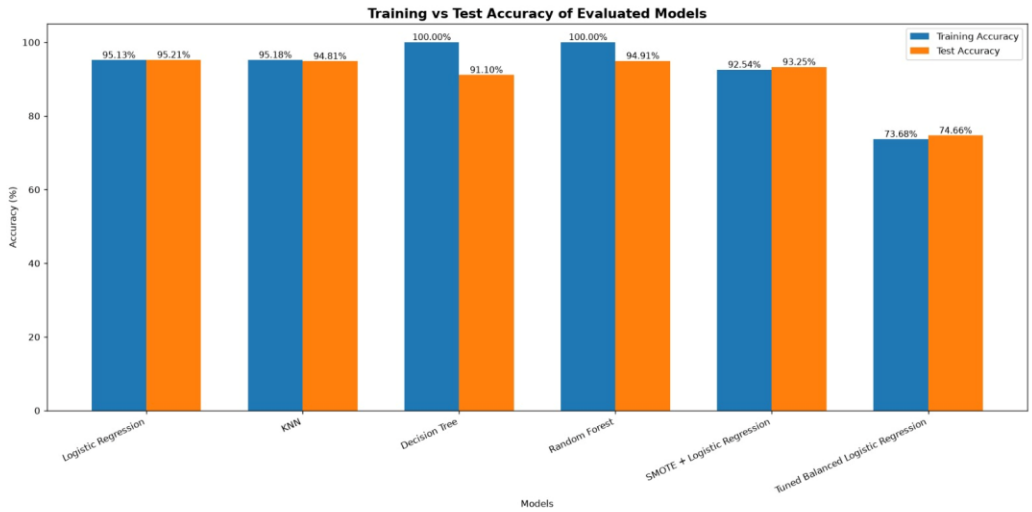


Figure 9.2: Training and Test Accuracy Comparison of Evaluated Models

The training and test accuracy comparison shows that Logistic Regression, KNN and Tuned Balanced Logistic Regression have relatively small differences between training and test performance. Decision Tree and Random Forest achieve 100% training accuracy but lower test accuracy, indicating a larger generalization gap and possible overfitting. The Tuned Balanced Logistic Regression model shows a small difference between training accuracy (73.68%) and test accuracy (74.66%), indicating consistent performance on unseen test data.

9.2 Best Model: Tuned Balanced Logistic Regression

Tuned Balanced Logistic Regression was selected as the final model for the MedInsight stroke-risk prediction system. The model was chosen after comparing multiple classifiers and applying class balancing and hyperparameter tuning. Although some models achieved higher overall accuracy, the tuned balanced model achieved a recall of 80.00%, indicating a stronger ability to identify actual stroke cases. Since identifying potential stroke cases is particularly important in a screening-oriented system, recall was given greater importance during final model selection.

Final Model Performance

Metric	Value
Accuracy	74.66%
Precision	13.84%
Recall	80.00%
F1-score	23.60%
ROC-AUC	82.80%

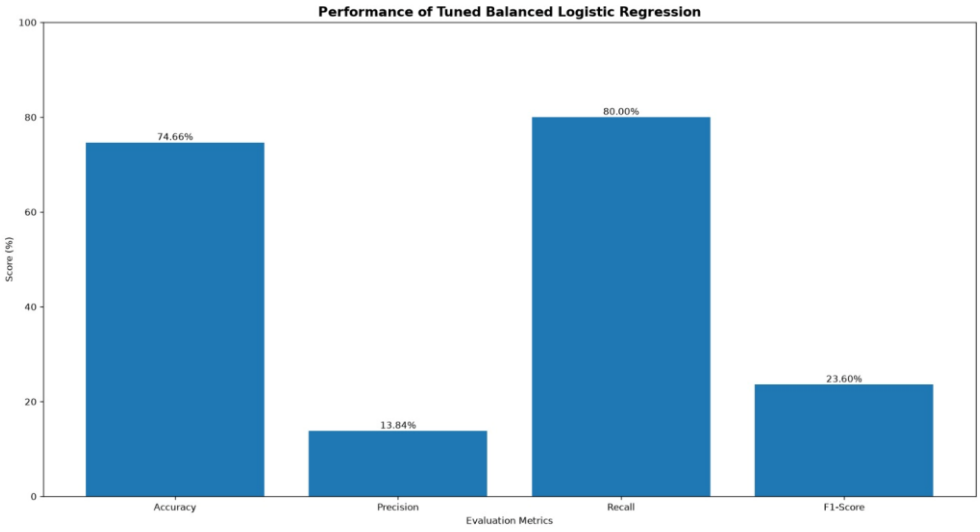


Figure 9.3: Performance of the Tuned Balanced Logistic Regression

The final model prioritizes the identification of stroke cases rather than maximizing overall accuracy. Its **80.00% recall** means that most of the actual stroke cases in the test set were correctly identified. The lower precision indicates that the model also produces a number of false positive predictions, which is an important consideration when interpreting the screening results.

The model achieved an accuracy of 74.66%, precision of 13.84%, recall of 80.00%, F1-score of 23.60%, and ROC-AUC of 82.80%. The ROC-AUC value indicates the model's ability to distinguish between stroke and non-stroke cases across different classification thresholds.

9.3 Confusion Matrix Analysis

The confusion matrix was used to evaluate the classification performance of the final Tuned Balanced Logistic Regression model by comparing the predicted outcomes with the actual stroke labels. It summarizes the number of True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN). This analysis provides a clearer understanding of the model's ability to identify stroke cases and non-stroke cases.

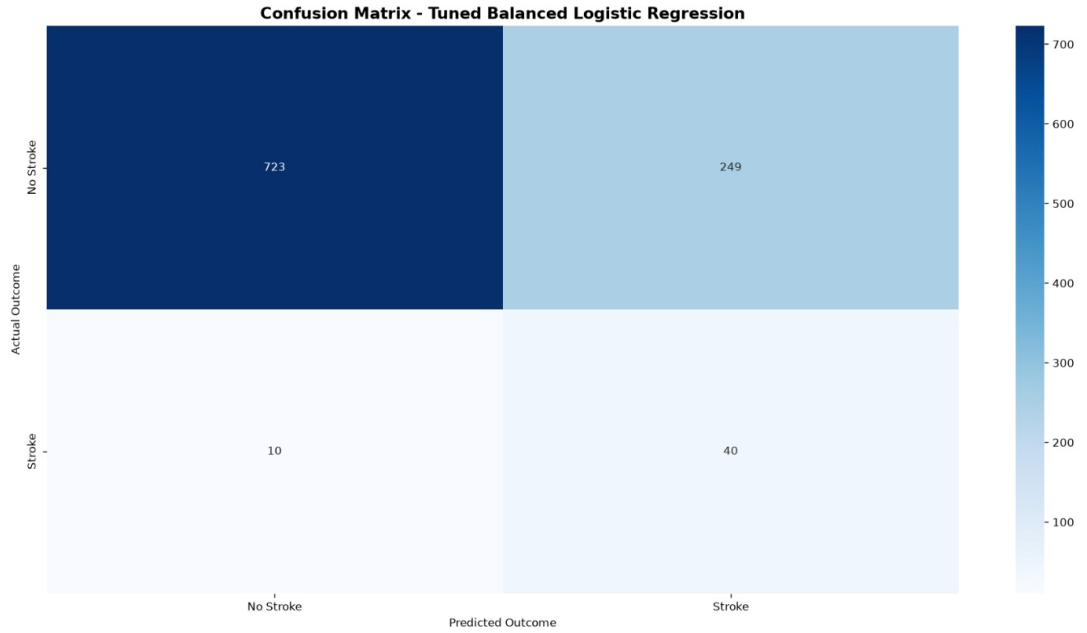


Figure 9.4: Confusion Matrix of Tuned Balanced Logistic Regression

The confusion matrix shows that the model correctly identified **40 stroke cases** and **723 non-stroke cases**. It incorrectly classified **10 actual stroke cases as non-stroke**, while **249 non-stroke cases were classified as stroke**. The relatively small number of false negatives contributes to the model's high recall of **80.00%**, which supports its use as a screening-oriented model where identifying potential stroke cases is important.

9.4 Prediction Test Cases

9.4.1 Test Case 1 - Higher Stroke Risk Prediction

Test Case 1 demonstrates the prediction generated by the MedInsight screening system for a sample patient. The system classified the patient as having a **higher estimated stroke risk** with an estimated stroke probability of **65.40%**. The prediction rationale further shows the contribution of individual patient features to the model output. In this case, age and hypertension were among the features that increased the predicted output, while some other features contributed negatively.

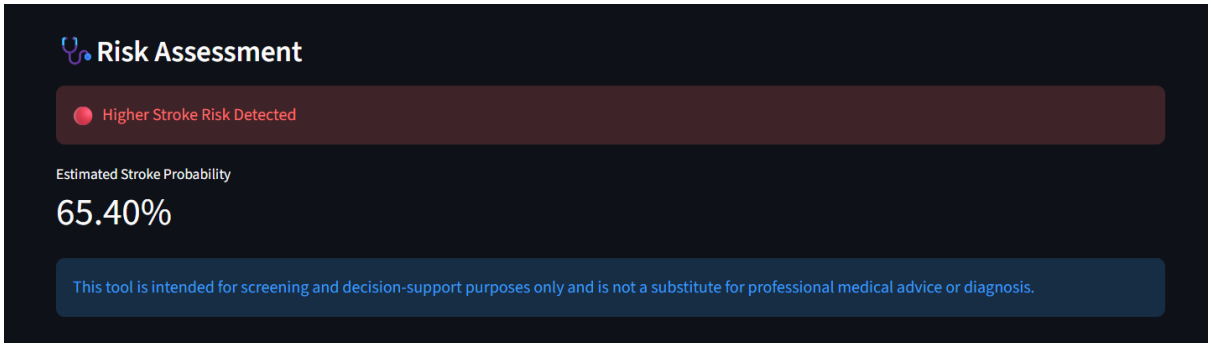


Figure 9.5: Test Case 1 - Higher Stroke Risk Assessment

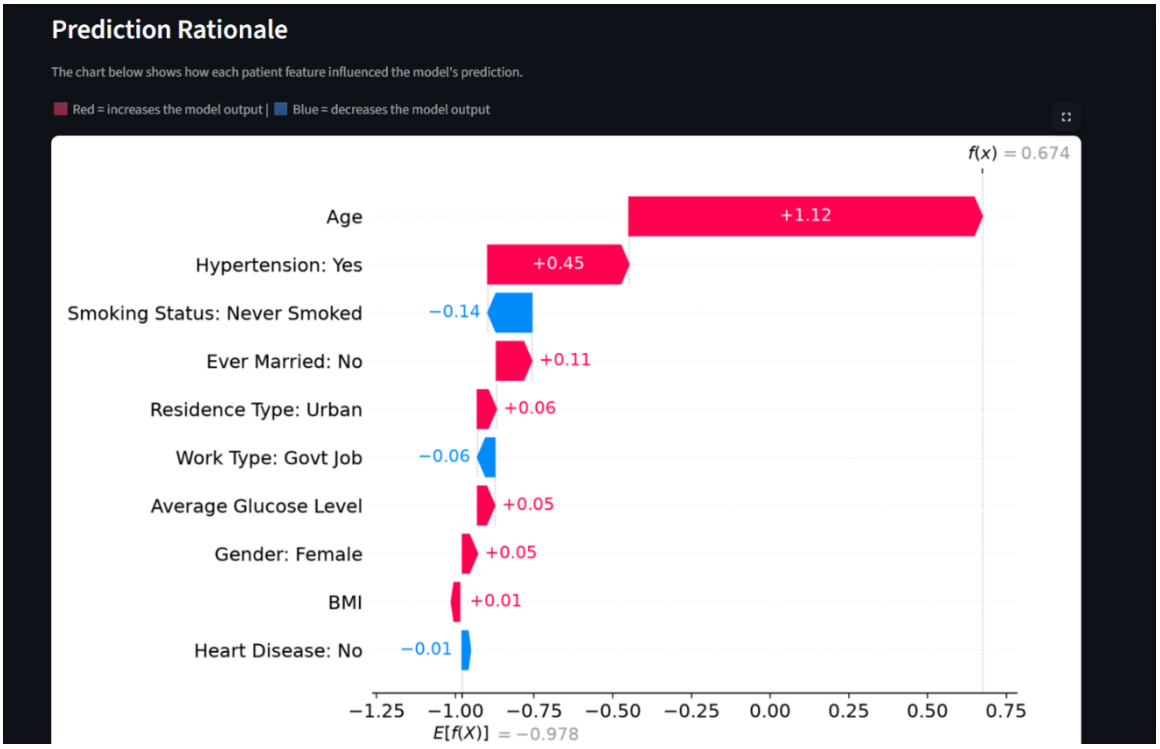


Figure 9.6: Prediction Rationale for Test Case 1

9.4.2 Test Case 2 - Lower Stroke Risk Prediction

Test Case 2 demonstrates the prediction generated by the MedInsight screening system for a different sample patient profile. In this case, the system indicates a lower estimated stroke risk based on the information provided by the user. The prediction rationale also presents the contribution of individual patient features to the model output, providing an interpretable explanation of the generated result.

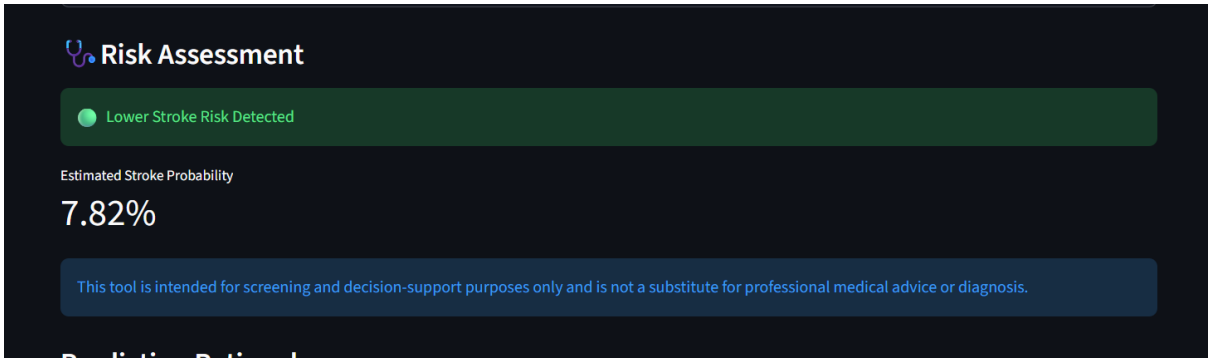


Figure 9.7: Test Case 2 - Lower Stroke Risk Assessment

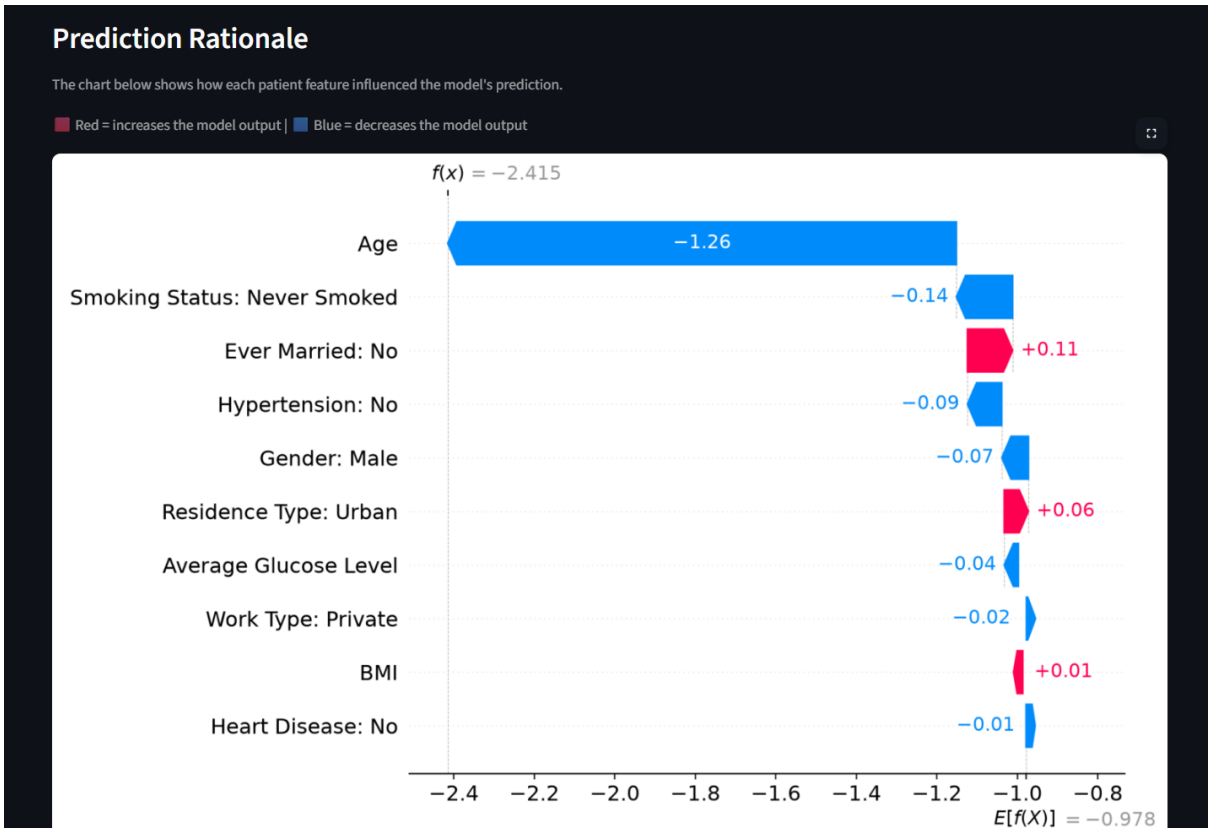


Figure 9.8: Prediction Rationale for Test Case 2

The second test case demonstrates that the system can produce a different risk assessment for a different combination of patient characteristics. The accompanying feature contribution explanation provides insight into the factors that increase or decrease the model output.

9.5 Result Interpretation

9.5.1 Effect of Dataset Construction

The construction and characteristics of the stroke dataset have a significant influence on the performance of the machine learning models. The dataset contains patient related demographic, medical and lifestyle features such as age, gender, hypertension, heart disease, average glucose level, BMI, smoking status, work type and residence type. These features provide information about different factors associated with stroke risk. However, the dataset contains a considerably larger number of non-stroke cases than stroke cases, resulting in class

imbalance. This imbalance affects model learning because a model may achieve high overall accuracy while failing to identify a sufficient number of actual stroke cases. Therefore the dataset characteristics and class distribution were important considerations during model evaluation and selection.

Key Observation:
Class imbalance strongly affects the interpretation of accuracy and makes **Recall** an important metric for evaluating the stroke prediction models.

9.5.2 Feature Relevance

The features used in the MedInsight system provide important information for estimating stroke risk. Features such as **age, average glucose level, hypertension, heart disease, BMI and smoking status** represent different demographic, medical and lifestyle factors that can influence the model's prediction. The SHAP-based explanations generated by the system provide further insight into how individual features contribute to a particular prediction. Among the observed features, age showed a strong contribution to the model output, while factors such as **hypertension and average glucose level** also influenced the predicted risk. This demonstrates that the model considers multiple patient characteristics rather than relying on a single feature when generating a prediction.

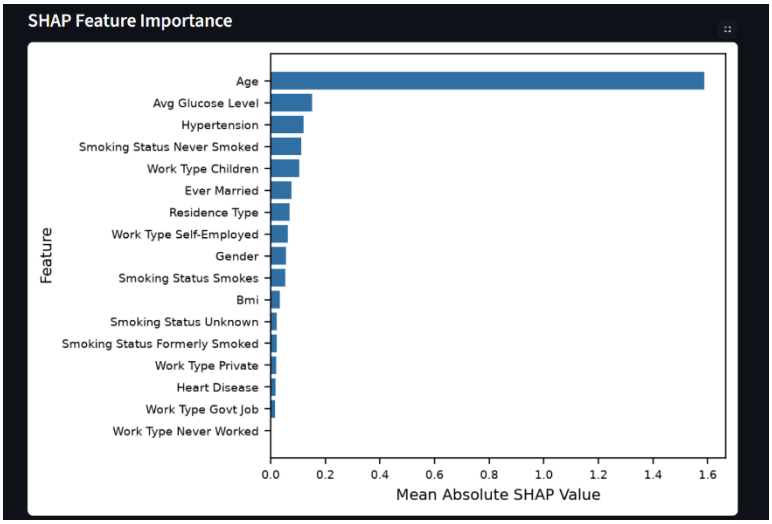


Figure 9.9: SHAP Feature Importance of the Final Model

9.5.3 Effect of Class Imbalance and Model Balancing

Class imbalance had a noticeable effect on the performance of the evaluated models. The dataset contains considerably more non-stroke cases than stroke cases, which can cause a model to favor the majority class and achieve high overall accuracy while identifying very few actual stroke cases. This pattern was observed in several of the evaluated models, where accuracy was above 90% but recall remained very low. To address this issue, class balancing techniques were evaluated, including SMOTE and balanced class weighting. The Balanced Logistic Regression model achieved a recall of 80.00%, compared with only 2.00% recall for

the conventional Logistic Regression model. Although the balanced model had a lower accuracy of 74.66%, it was considerably more effective at identifying actual stroke cases.

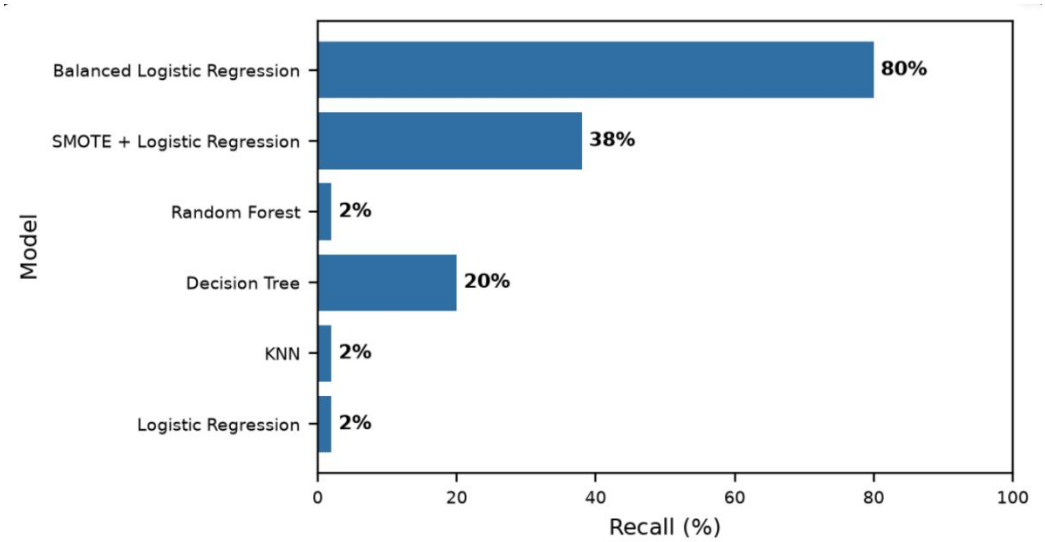


Figure 9.10: Recall Comparison of Evaluated Models

Key Observation:
For stroke-risk prediction a high accuracy score alone does not indicate good performance. Improving recall is important because false negative predictions may cause actual stroke cases to be missed.

9.5.4 Possible Reasons for Model Performance

The differences in performance among the evaluated machine learning models can be attributed to their different learning mechanisms and their response to the imbalanced stroke dataset. Logistic Regression, KNN, Decision Tree and Random Forest achieved relatively high accuracy but showed low recall, indicating that they were more likely to classify cases as the majority non-stroke class. The use of SMOTE with Logistic Regression improved recall to **38.00%**, while Balanced Logistic Regression further increased recall to **80.00%** by assigning greater importance to the minority stroke class. The final performance therefore reflects both the characteristics of the dataset and the balancing strategy used during model training.

Key Observation:
The balancing strategy had a major effect on the model's ability to identify stroke cases, making Balanced Logistic Regression more suitable for the project's screening objective.

9.5.5 Limitations of Model Evaluation

Although the developed MedInsight system demonstrates the ability to identify potential stroke cases, several limitations should be considered when interpreting the model results. First, the dataset contains a considerable class imbalance between stroke and non-stroke cases, which can affect model performance and the reliability of accuracy as a standalone evaluation metric. Second, the relatively low precision of the selected Tuned Balanced Logistic Regression model indicates that a number of non-stroke cases are classified as potential stroke cases. Third, the model was evaluated using a specific dataset and test set and its performance may vary when applied to different populations or larger and more diverse datasets. Therefore the predictions generated by the system should be considered as screening and decision-support results rather than a medical diagnosis.

Key Observation:
Model performance should be interpreted using multiple evaluation metrics and within the limitations of the dataset. The MedInsight system is intended for screening and decision support purposes and should not replace professional medical diagnosis.

CHAPTER 10 - ADVANTAGES AND LIMITATIONS

10.1 Advantages

❖ **User Authentication:**
The Streamlit application includes a login and authentication mechanism before access to the MedInsight dashboard, providing a controlled entry point to the application.

❖ **Automated Stroke-Risk Screening:**
The system automatically processes patient demographic, medical and lifestyle information and generates a predicted stroke outcome together with an estimated stroke probability.

❖ **Comprehensive Model Comparison:**
Six machine learning approaches (Logistic Regression, KNN, Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression) are trained and compared using multiple evaluation metrics.

❖ **Class Imbalance Handling:**
The project evaluates SMOTE and class weighted Logistic Regression to address the imbalance between stroke and non-stroke cases, improving the identification of minority class stroke cases.

❖ **Recall-Focused Model Selection:**
The project gives particular importance to recall because identifying actual stroke cases is important for a screening-oriented system. The selected Tuned Balanced Logistic Regression achieved **80.00% recall**.

❖ **Hyperparameter Tuning:**
The best performing balanced Logistic Regression model is further tuned before being selected as the final prediction model, improving the model selection process.

❖ **Explainable Predictions:**
The system uses SHAP-based explanations to show how individual patient features contribute to a prediction, while also providing global feature importance analysis.

❖ **Comprehensive Model Analytics:**
The system provides a dedicated analytics section containing model performance comparison, recall comparison, confusion matrix, correlation heatmap, ROC/AUC analysis and SHAP feature analysis.

❖ **User-Friendly Streamlit Application:**
The trained Tuned Balanced Logistic Regression model is integrated into an interactive Streamlit application with automated input preprocessing and feature preparation before prediction.

❖ **Risk Assessment and Personalised Guidance:**
The application presents the predicted outcome and estimated probability, provides a lower/moderate/higher risk assessment and gives personalised screening guidance based on relevant risk factors and health recommendations.

❖ **Downloadable Assessment Report:**
The Streamlit application provides an option to download the generated assessment as a PDF report, making the prediction results easier to retain and review.

❖ **Support for Repeated Predictions:**
After completing an assessment, the system is designed to become ready for a new input and the next prediction, allowing repeated use of the screening application.

10.2 Limitations

❖ **Dataset Size and Diversity:**
The model is trained and evaluated on a single stroke dataset containing **5110 patient records**. The dataset may not fully represent the diversity of patient populations encountered in real-world healthcare settings.

❖ **Class Imbalance:**
The dataset contains substantially more non-stroke cases than stroke cases. This imbalance can cause conventional models to favour the majority class and achieve high accuracy while identifying very few actual stroke cases.

❖ **High False-Positive Rate:**
Although the selected Tuned Balanced Logistic Regression achieved **80.00% recall**, its precision was only **13.84%**, indicating that a considerable number of non-stroke cases were classified as potential stroke cases.

❖ **Limited Clinical Information:**
The prediction is based on the features available in the dataset, such as age, hypertension, heart disease, glucose level, BMI, smoking status and other demographic or lifestyle attributes. It does not include additional clinical information such as medical examination results, imaging data or laboratory investigations beyond the available features.

❖ **Limited External Validation:**
The model was evaluated using a particular train-test split and the available dataset. Its performance may differ when applied to a larger, more diverse or independently collected dataset.

❖ **Screening Rather Than Diagnosis:**
The system provides an estimated stroke risk prediction and is intended for screening and decision support purposes. It cannot replace professional medical examination, clinical diagnosis or treatment decisions.

CHAPTER 11 – CONCLUSION

This project aimed to develop and evaluate a Machine Learning-based system for stroke-risk prediction using demographic, medical and lifestyle related patient information. The problem was formulated as a binary classification task, where the system predicts whether a patient is likely to be at risk of stroke. Multiple supervised learning approaches, including Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression, were implemented, trained and compared. The objective was to identify a suitable model for stroke-risk screening and integrate the selected model into an interactive Streamlit based prediction system.

Dataset and Preprocessing:

The stroke dataset contains patient-related demographic, medical and lifestyle features along with a binary stroke target variable. The preprocessing stage included data inspection, handling of categorical variables, feature preparation, train-test splitting and feature scaling where required by the selected algorithms. Since the dataset contained a substantial imbalance between non-stroke and stroke cases, class balancing approaches such as SMOTE and class-weighted Logistic Regression were also evaluated. These preprocessing and preparation steps provided a consistent basis for training and comparing the machine learning models.

Models and Evaluation:

Six machine learning approaches (Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, SMOTE + Logistic Regression and Balanced Logistic Regression) were implemented and evaluated using the same dataset. Model performance was assessed using Accuracy, Precision, Recall and F1-score with additional analysis through the confusion matrix and training-versus-test accuracy comparison. The evaluation showed that conventional models could achieve high overall accuracy while producing very low recall for stroke cases because of the class imbalance. Therefore multiple evaluation metrics were considered rather than relying on accuracy alone.

Best Model and Its Interpretation:

After comparing the evaluated approaches and applying class balancing and hyperparameter tuning, Tuned Balanced Logistic Regression was selected as the final model for the MedInsight system. The model achieved an accuracy of **74.66%**, precision of **13.84%**, recall of **80.00%** and an F1-score of **23.60%**. Although its overall accuracy was lower than several conventional models, its substantially higher recall made it more suitable for the project's screening-oriented objective, as it was able to identify a greater proportion of actual stroke cases. SHAP-based analysis was also used to provide insight into the contribution of individual features to model predictions.

Prediction System:

The final trained model was integrated into an interactive Streamlit application named MedInsight. The system allows users to enter relevant patient information and generates an estimated stroke-risk prediction and probability. It also provides prediction rationale through feature-contribution analysis, risk assessment information and general risk-factor guidance. The application includes model analytics and provides a downloadable assessment report, making the developed system a complete machine-learning based screening and decision-support prototype.

Overall Achievement

• Complete End-to-End Machine Learning Pipeline:
A complete workflow was developed covering data loading, preprocessing, feature preparation, train-test splitting, model training, evaluation, class balancing, hyperparameter tuning and final prediction.

• Comparative Evaluation of Multiple Models:
Six machine learning approaches were implemented and compared, allowing the project to identify a model that was better aligned with the screening objective rather than selecting a model based only on accuracy.

• Handling of Class Imbalance:
The project investigated the effect of class imbalance and evaluated SMOTE and balanced Logistic Regression approaches to improve the identification of minority-class stroke cases.

• Explainable Prediction:
SHAP-based analysis was incorporated to explain feature contributions and provide greater interpretability of both overall model behavior and individual predictions.

• Interactive Prediction Application:
The final model was deployed through a Streamlit interface that accepts patient information and generates an estimated stroke-risk assessment.

• Comprehensive Result Analysis:
The project included model comparison, confusion matrix analysis, training-versus-test accuracy analysis, test cases and feature-importance analysis to provide a broader evaluation of the developed system.

• Practical Screening-Oriented Application:
The project demonstrates how machine learning can be integrated with an interactive application to support preliminary stroke-risk screening and decision-support.

Practical Usefulness

The developed MedInsight system demonstrates the potential practical usefulness of machine learning for preliminary stroke-risk screening. By combining patient information with a trained classification model, the system can provide a rapid estimated risk assessment and an explanation of important contributing factors. With further validation using larger and more diverse clinical datasets and appropriate healthcare integration such a system could potentially support healthcare professionals as a decision-support tool for preliminary risk assessment. However the system is not intended to replace professional medical diagnosis or clinical decision making.

Important Limitations

The project has several limitations including the substantial class imbalance present in the dataset, the relatively low precision of the selected model and the dependence on the available dataset features. The model was evaluated using a specific dataset and test set and its performance may vary when applied to different populations or independently collected clinical data. Furthermore the system has not undergone real world clinical validation. Therefore the current implementation should be considered a machine learning based screening and decision-support prototype rather than a clinically validated diagnostic system.

CHAPTER 12 – FUTURE SCOPE

Building on the limitations identified in Chapter 10 and the results discussed in Chapter 9, several directions for future improvement and extension of the MedInsight stroke-risk prediction system are proposed below. These improvements are intended as possible future work and are not part of the current implementation.

Possible Future Improvement: Larger and More Diverse Clinical Dataset

The most important future improvement is to train and validate the system using larger and more diverse stroke datasets collected from multiple hospitals and patient populations. A broader dataset could improve the generalization of the model and provide a more reliable assessment of its performance across different populations.

Additional Proposed Improvements

❖ **Larger Dataset Collection:**
Collecting more patient records from different healthcare institutions and demographic groups could reduce dataset related bias and improve the generalization of the prediction model.

❖ **External Validation:**
The final model can be evaluated on an independent dataset that was not involved in model development. This would provide a stronger assessment of how well the system performs on unseen patient populations.

❖ **Improved Class Imbalance Handling:**
Future work can investigate additional class balancing techniques and compare different strategies for improving minority class detection while reducing false positive predictions.

❖ **Advanced Machine Learning Models:**
More advanced ensemble and machine learning approaches can be explored to determine whether they can provide a better balance between recall, precision and overall predictive performance.

❖ **Deep Learning Approaches:**
Deep learning models can be investigated using larger datasets to determine whether they can learn more complex relationships among demographic, medical and lifestyle related features.

❖ **Advanced Feature Engineering:**
Additional meaningful features and feature interactions can be investigated to improve the model's ability to capture relationships between factors associated with stroke risk.

❖ **Additional Clinical Features:**
Future versions can incorporate additional clinically relevant information, such as more detailed medical history, laboratory measurements and other validated risk factors, subject to data availability and privacy requirements.

❖ Advanced Hyperparameter Optimization:
Although hyperparameter tuning was already applied to the final Balanced Logistic Regression model, future work can explore more extensive optimization strategies such as **Randomized Search or Bayesian optimization** and apply them systematically across additional models to identify potentially better parameter configurations.

❖ Cross-Validation Strategy:
Instead of relying primarily on a single train-test split, Stratified K-Fold cross-validation can be used to obtain more reliable estimates of model performance while preserving the class distribution across folds.

❖ Enhanced Explainable AI:
The existing SHAP-based explanation system can be expanded with additional global and local interpretability visualizations, allowing users to understand both overall feature importance and individual prediction behaviour more clearly.

❖ Real-Time and Integrated Data Input:
Future versions could integrate the system with structured healthcare data sources or electronic health records, subject to appropriate security, privacy and authorization requirements, reducing the dependence on manual data entry.

❖ Cloud Deployment and API Integration:
The prediction model could be deployed through a secure cloud based API so that authorized healthcare applications could access the prediction service without requiring the complete application to run locally.

❖ Clinical Validation Study:
Before any real world healthcare use, the system should undergo extensive validation with healthcare professionals and appropriately collected clinical data. Clinical evaluation would be necessary to assess its reliability, usefulness and safety in practical settings.

❖ Continuous Model Monitoring:
A future deployment could include monitoring of model performance and data distribution over time so that changes in patient populations or data characteristics can be identified and the model can be retrained when appropriate.

CHAPTER 13 – REFERENCES

The following references include the software libraries, machine learning documentation, dataset source and explainability/deployment tools used in the development of the MedInsight Stroke-Risk Prediction System.

1. Python Software Foundation. *Python Documentation.* Available at: https://docs.python.org/3/

2. Scikit-learn Developers. *Scikit-learn Documentation.* Available at: https://scikit-learn.org/stable/

3. Pandas Development Team. *Pandas Documentation.* Available at: https://pandas.pydata.org/docs/

4. NumPy Developers. *NumPy Documentation.* Available at: https://numpy.org/doc/stable/

5. Matplotlib Development Team. *Matplotlib Documentation.* Available at: https://matplotlib.org/stable/

6. Seaborn Developers. *Seaborn Documentation.* Available at: https://seaborn.pydata.org/

7. Scikit-learn Developers. *Logistic Regression — User Guide.* Available at: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

8. Scikit-learn Developers. *K-Nearest Neighbors — User Guide.* Available at: https://scikit-learn.org/stable/modules/neighbors.html

9. Scikit-learn Developers. *Decision Trees — User Guide.* Available at: https://scikit-learn.org/stable/modules/tree.html

10. Scikit-learn Developers. *Random Forest — User Guide.* Available at: https://scikit-learn.org/stable/modules/ensemble.html#random-forests

11. Scikit-learn Developers. *Model Evaluation and Metrics.* Available at: https://scikit-learn.org/stable/modules/model_evaluation.html

12. Scikit-learn Developers. *Preprocessing and StandardScaler.* Available at: https://scikit-learn.org/stable/modules/preprocessing.html

13. imbalanced-learn Developers. *SMOTE — Synthetic Minority Over-sampling Technique.* Available at: https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html

14. SHAP Developers. *SHAP Documentation — SHapley Additive exPlanations.* Available at: https://shap.readthedocs.io/

15. Streamlit Developers. *Streamlit Documentation.* Available at: https://docs.streamlit.io/

16. Kaggle. *Stroke Prediction Dataset.* Available at: https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

NOTE: The references listed above include the official documentation, dataset source, machine learning libraries, tools and resources used during the development of the MedInsight Stroke-Risk Prediction System.